

DevOps

Tutorials

Theophilus Siameh

Outline

- Linux basics
- Application basics: Java, Python, Bash, SQL
- YAML files Intro
- Git Intro
- Jenkins Intro
- Jenkins Pipeline Intro
- Containers: Docker
- Orchestration: Kubernetes
- Microservices Intro
- Configuration: Ansible
- Infrastructure as Code: Terraform
- Python Basics
- Cloud: AWS Cloud, Azure
- Master any CLI commands
- Learn to install open source software
- Observability: Elasticsearch, Logstash, Kibana, Grafana
- Project: 3-4 examples
- Certification:
 - * AWS Certified Developer-Associate,
 - * AWS Certified DevOps Engineer-Professional,
 - * AWS Certified Security-Professional

Tools to Install

- <https://jupyter.org/>
- <https://www.jetbrains.com/pycharm/download/?section=mac>
- <https://code.visualstudio.com/>
- <https://www.python.org/downloads/>
- <https://docs.docker.com/desktop/install/mac-install/>
- <https://minikube.sigs.k8s.io/docs/start/>
- <https://github.com>
- <https://console.amazon.com>
-

DevOps - 50 Linux Commands



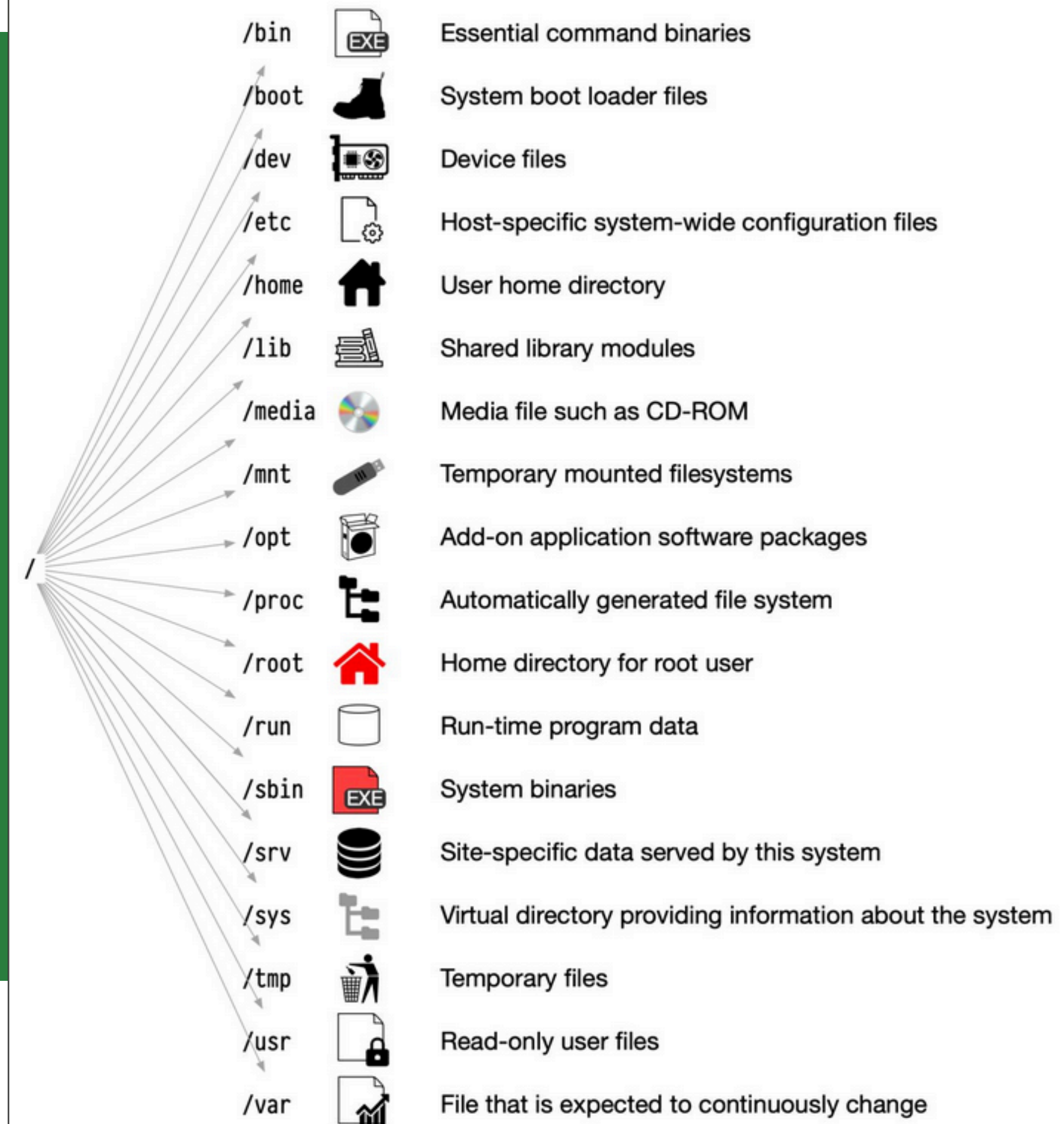
Top 50 Linux Commands you must know



1.is	1.clear	1.diff	1.kill and killall	1.apt, pacman, yum, rpm
2.pwd	2.echo	2.cmp	2.df	2.sudo
3.cd	3.less	3.comm	3.mount	3.cal
4.mkdir	4.man	4.sort	4.chmod	4.alias
5.mv	5.unman	5.export	5.chown	5.dd
6.cp	6.whoami	6.zip	6.ifconfig	6.whereis
7.rm	7.tar	7.unzip	7.traceroute	7.whatis
8.touch	8.grep	8.ssh	8.wget	8.top
9.in	9.head	9.service	9.ufw	9. useradd
10.cat	10.tail	10.ps	10.iptables	10.passwd

Linux File Systems

ByteByteGo.com

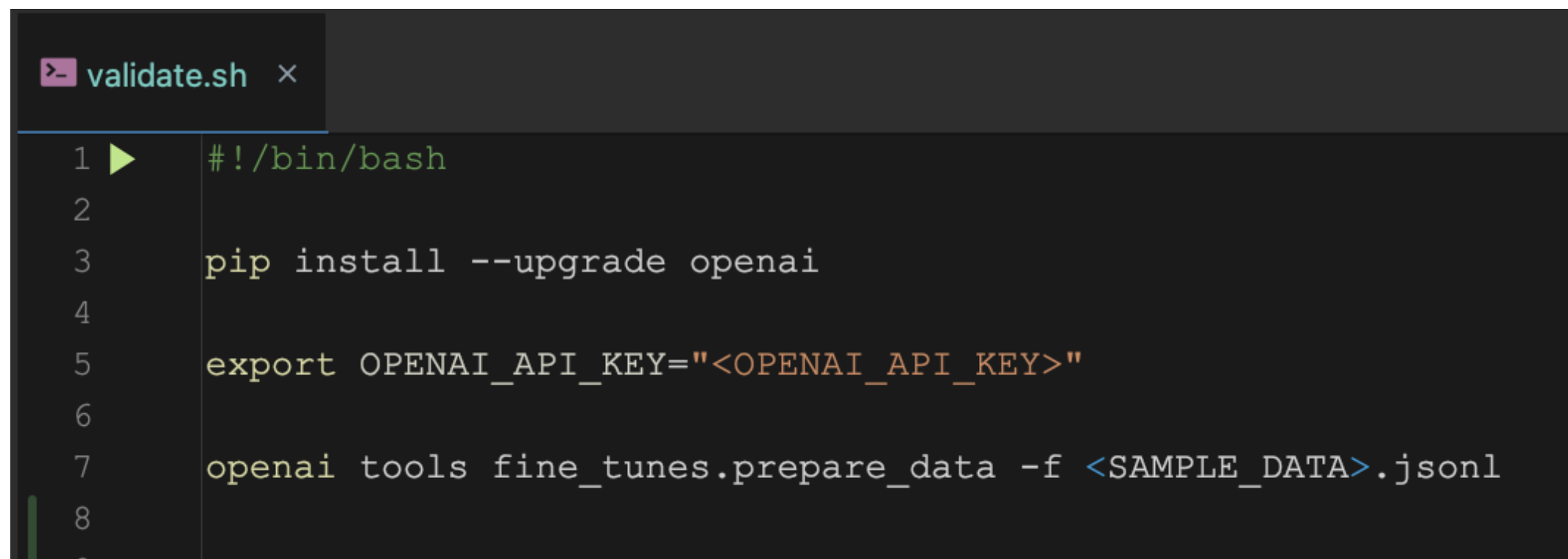


Shell Scripting

- 1 Write a shell script function to find and kill all the zombie processes.
- 2 Find the first 10 biggest files in the file system and write the output to a file.
- 3 Shell script to gracefully unmount a disk.
- 4 shell script to send email
- 5 Shell script to monitor CPU, Memory, and Disk usage and send the output to a file in table format and send an alert if either of them exceeds a certain threshold.
- 6 Shell script to find the files created and their sizes. It should accept the number of days as input. Or a from and to date format as inputs.
- 7 Write a shell script to automate the process of creating new user accounts on a Linux server and setting up their permissions and SSH access.
- 8 Write a shell script to the list of users logged in by date and write it to an output file.
- 9 Shell script to Copy files recursively to remote hosts
- 10 Shell script that displays the number of failed login attempts by IP address and location.
- 11 Shell script parses a log file and forwards a specific value with a timestamp to an output file.
- 12 Write a shell script to automate the process of rotating log files and compressing old files to save disk space.
- 13 Write a shell script to check the status of a list of URLs and send an email notification if any of them are down.
- 14 Write a shell script to automate the process of updating a list of servers with the latest security patches.

A shell is a type of computer program called a command-line interpreter that lets Linux and Unix users control their operating systems with command-line interfaces. Shells allow users to communicate efficiently and directly with their operating systems.

Extension: `.sh`



```
validate.sh x
1 ▶ #!/bin/bash
2
3   pip install --upgrade openai
4
5   export OPENAI_API_KEY="<OPENAI_API_KEY>"
6
7   openai tools fine_tunes.prepare_data -f <SAMPLE_DATA>.jsonl
8
9
```

DevOps - 50 Linux Commands

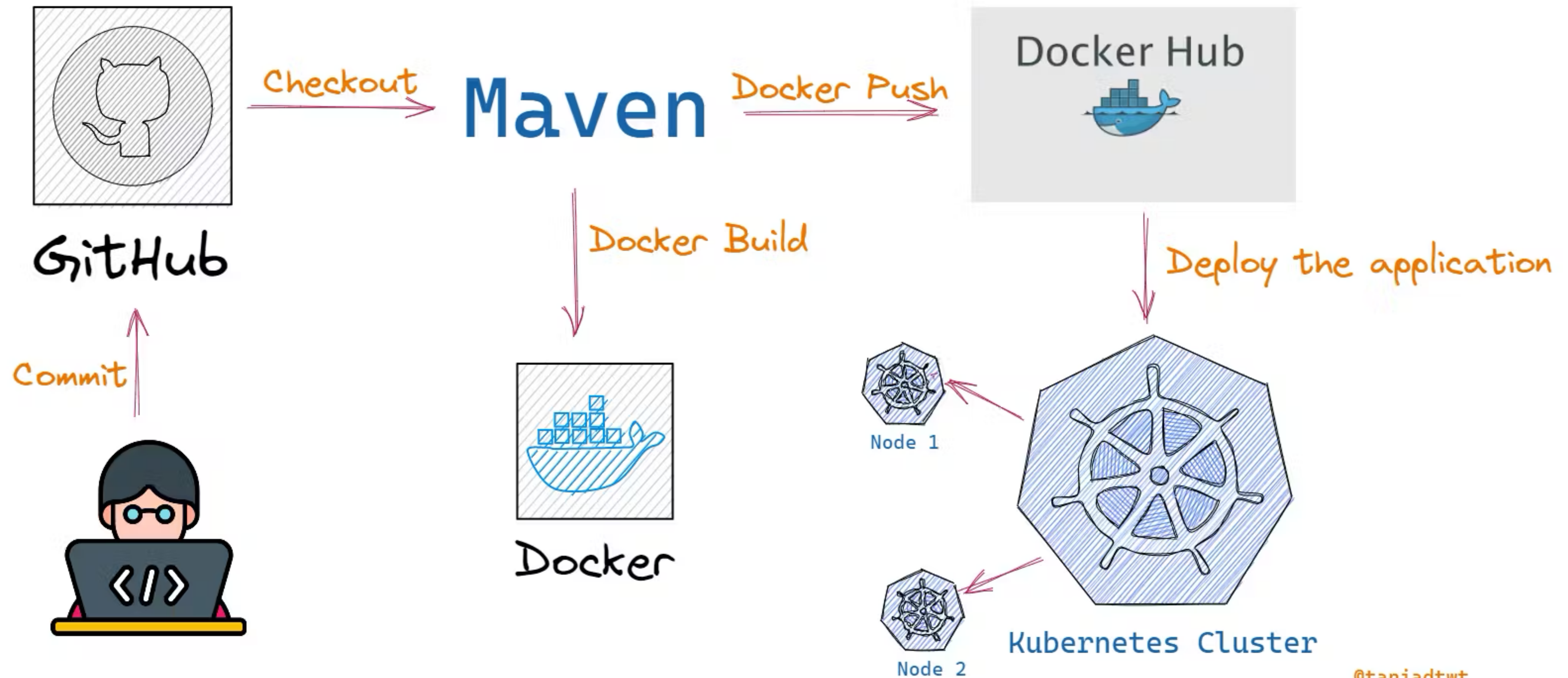
1. **ls** - The most frequently used command in Linux to list directories
2. **pwd** - Print working directory command in Linux
3. **cd** - Linux command to navigate through directories
4. **mkdir** - Command used to create directories in Linux
5. **mv** - Move or rename files in Linux
6. **cp** - Similar usage as mv but for copying files in Linux
7. **rm** - Delete files or directories
8. **touch** - Create blank/empty files
9. **ln** - Create symbolic links (shortcuts) to other files
10. **cat** - Display file contents on the terminal
11. **clear** - Clear the terminal display
12. **echo** - Print any text that follows the command
13. **less** - Linux command to display paged outputs in the terminal
14. **man** - Access manual pages for all Linux commands
15. **uname** - Linux command to get basic information about the OS
16. **whoami** - Get the active username
17. **tar** - Command to extract and compress files in Linux
18. **grep** - Search for a string within an output
19. **head** - Return the specified number of lines from the top
20. **tail** - Return the specified number of lines from the bottom
20. **tail** - Return the specified number of lines from the bottom
21. **diff** - Find the difference between two files
22. **cmp** - Allows you to check if two files are identical
23. **comm** - Combines the functionality of diff and cmp
24. **sort** - Linux command to sort the content of a file while outputting
25. **export** - Export environment variables in Linux
26. **zip** - Zip files in Linux
27. **unzip** - Unzip files in Linux
28. **ssh** - Secure Shell command in Linux
29. **service** - Linux command to start and stop services
30. **ps** - Display active processes
31. **kill and killall** - Kill active processes by process ID or name
32. **df** - Display disk filesystem information
33. **mount** - Mount file systems in Linux
34. **chmod** - Command to change file permissions
35. **chown** - Command for granting ownership of files or folders
36. **ifconfig** - Display network interfaces and IP addresses
37. **traceroute** - Trace all the network hops to reach the destination
38. **wget** - Direct download files from the internet
39. **ufw** - Firewall command
40. **iptables** - Base firewall for all other firewall utilities to interface with
41. **apt, pacman, yum, rpm** - Package managers depending on the distro
42. **sudo** - Command to escalate privileges in Linux
43. **cal** - View a command-line calendar
44. **alias** - Create custom shortcuts for your regularly used commands
45. **dd** - Majorly used for creating bootable USB sticks
46. **whereis** - Locate the binary, source, and manual pages for a command
47. **whatis** - Find what a command is used for
48. **top** - View active processes live with their system usage
49. **useradd and usermod** - Add new user or change existing users data
50. **passwd** - Create or update passwords for existing users

Package Managers

- Select a git repository and install on local system
 - * yum install ansible
 - * apt install ansible python3
- Services: they run in the background
 - * systemctl start <httpd>
 - * systemctl stop <httpd>
 - * systemctl status <httpd>
 - * systemctl enable <httpd>
 - * systemctl disable <httpd>

Application Basics

Workflow of Project



@taniadtwt

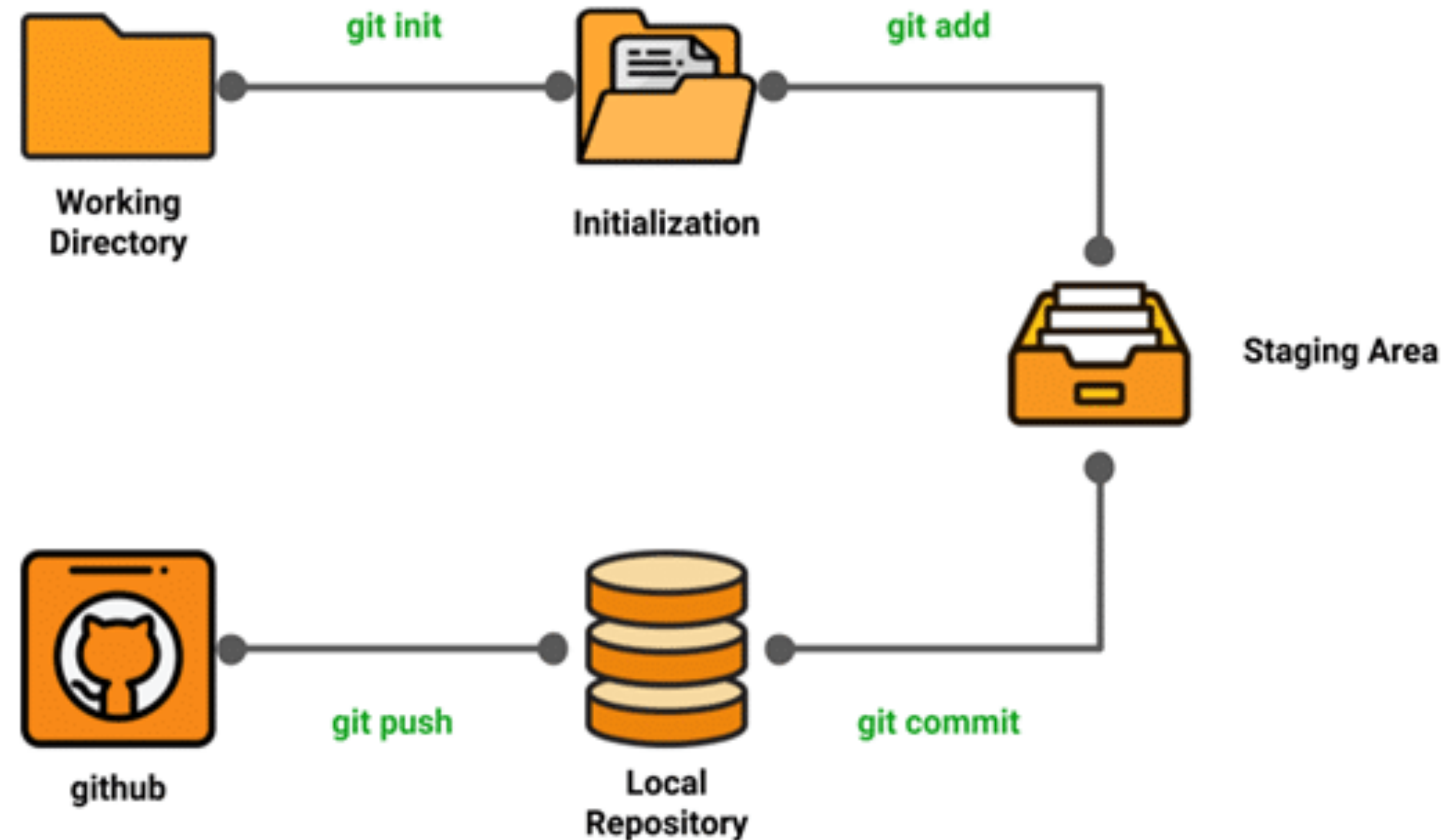
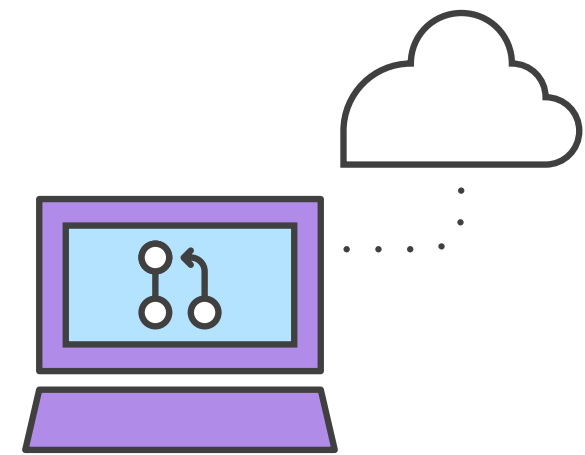
YAML Basics

```
---
doe: "a deer, a female deer"
ray: "a drop of golden sun"
pi: 3.14159
xmas: true
french-hens: 3
calling-birds:
  - huey
  - dewey
  - louie
  - fred
xmas-fifth-day:
  calling-birds: four
  french-hens: 3
  golden-rings: 5
  partridges:
    count: 1
    location: "a pear tree"
  turtle-doves: two
```

```
{
  "doe": "a deer, a female deer",
  "ray": "a drop of golden sun",
  "pi": 3.14159,
  "xmas": true,
  "french-hens": 3,
  "calling-birds": [
    "huey",
    "dewey",
    "louie",
    "fred"
  ],
  "xmas-fifth-day": {
    "calling-birds": "four",
    "french-hens": 3,
    "golden-rings": 5,
    "partridges": {
      "count": 1,
      "location": "a pear tree"
    },
    "turtle-doves": "two"
  }
}
```


GIT Basics

Git is a free and open-source version control system, originally created by Linus Torvalds in 2005.

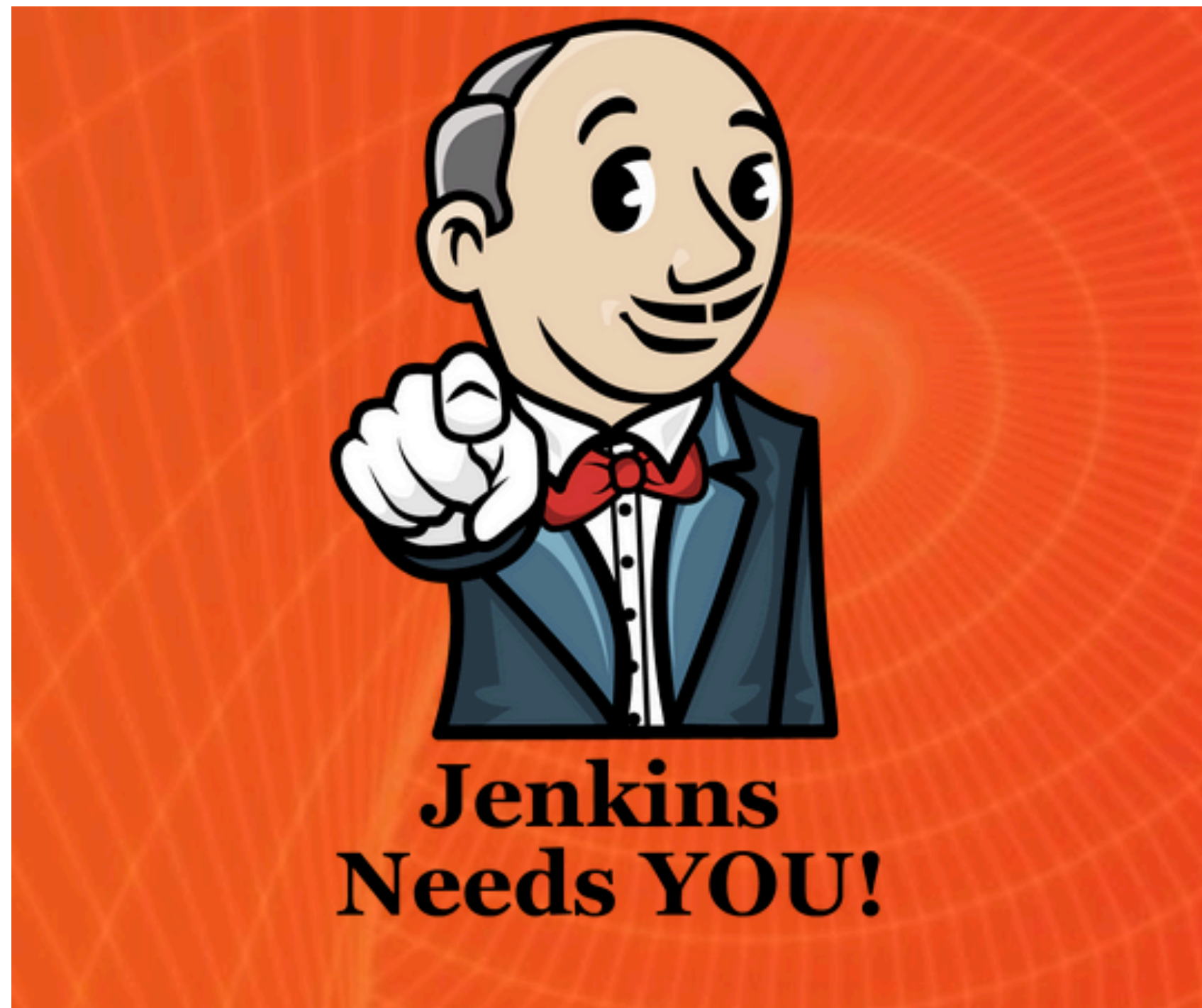


GitHub
GitLab
BitBucket

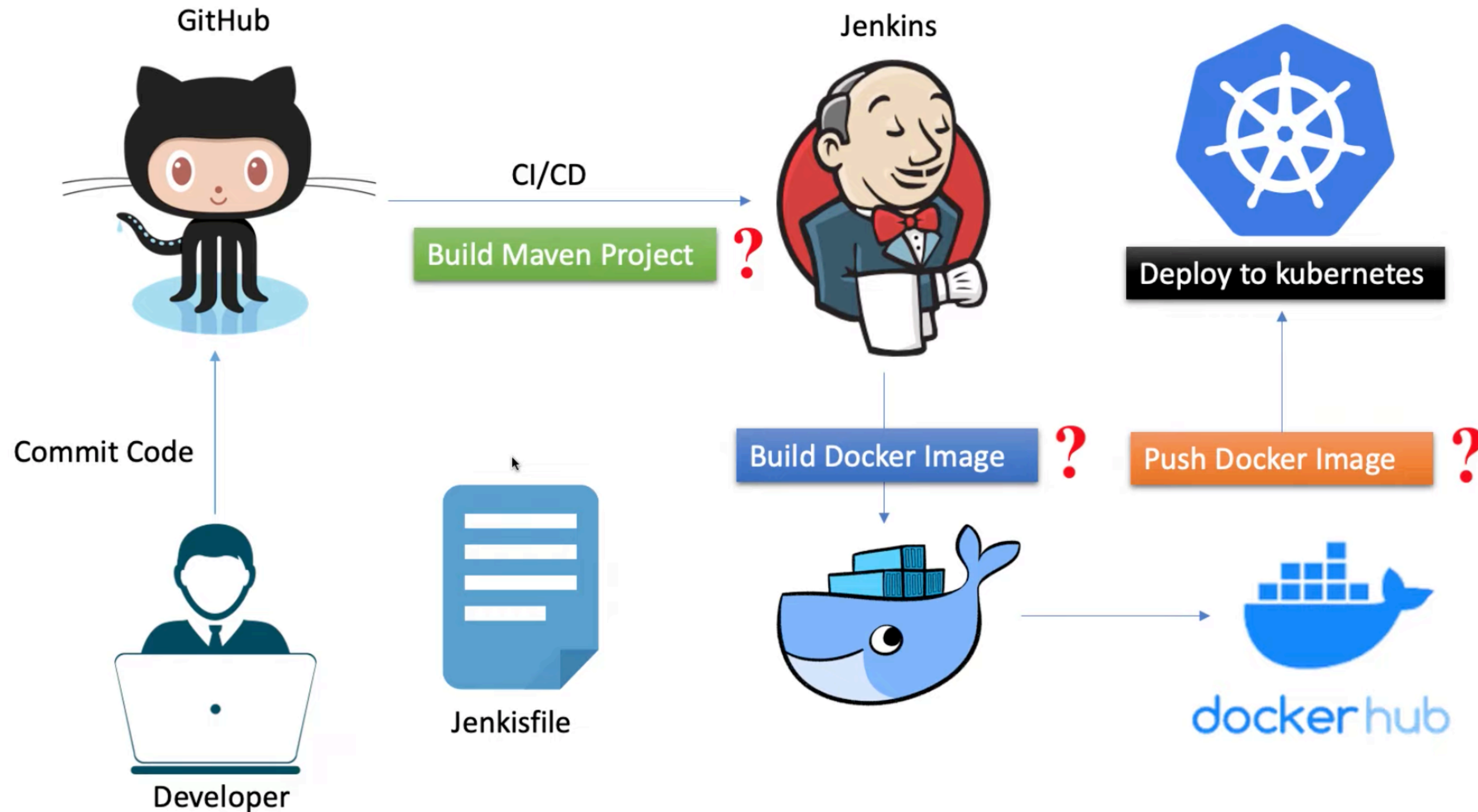
Jenkins Basics

Jenkins is a self-contained, open source automation server which can be used to automate all sorts of tasks related to building, testing, and delivering or deploying software.

Jenkins can be installed through native system packages, Docker, or even run standalone by any machine with a Java Runtime Environment (JRE) installed.



Jenkins Basics



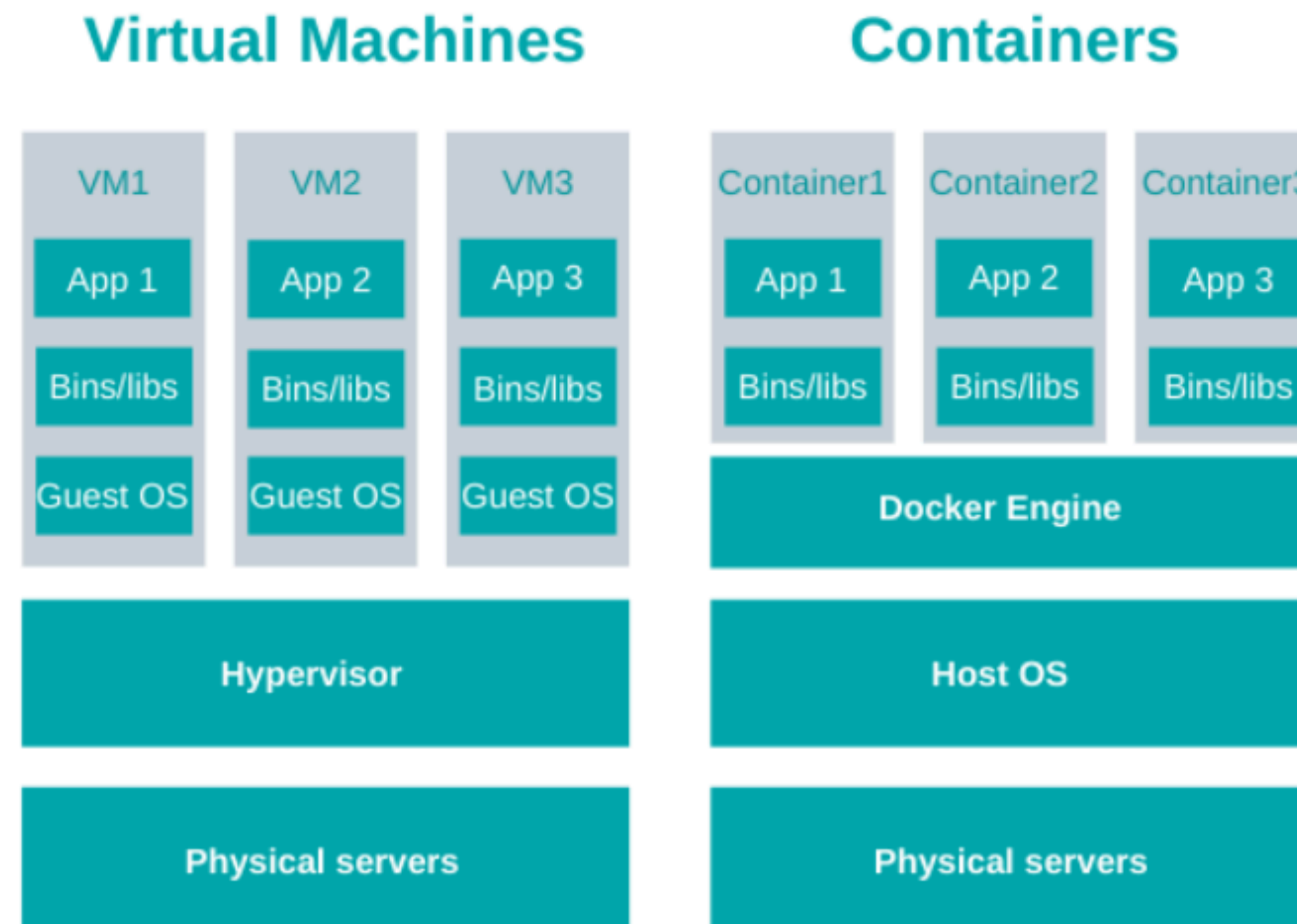
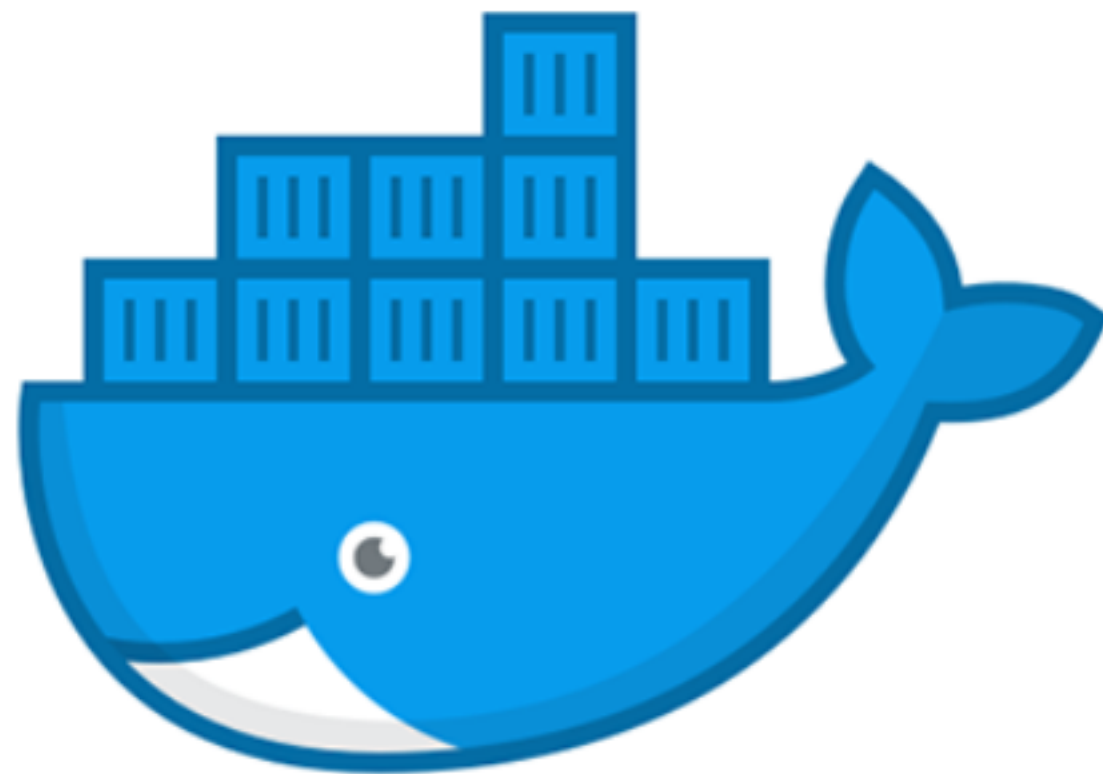
Docker Basics

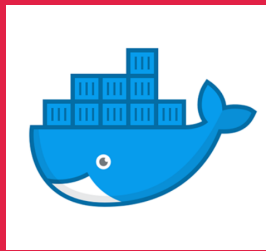
What is docker?

Docker is a tool designed to make it easier to create, deploy, and run applications by using containers.

Why docker?

Docker is useful to automate the deployment of applications inside a software containers, which makes the applications easy to ship and run virtually anywhere (i.e, platform independent). The Docker container processes run on the host kernel, unlike VM which runs processes in guest kernel.





Docker Basics

Accelerate how you build, share, and run modern applications.

18 million +
developers

7 million +
applications

13 billion +
monthly image downloads

Get started

See who uses Docker



<> Code Issues 615 Pull requests 215 Actions Security 1 Insights

 cli Public Watch 121 Fork 1.8k Star 4.1k

master

Go to file

Add file

<> Code

About

The Docker CLI

docker cli

Readme

Apache-2.0 license

Activity

4.1k stars

121 watching

1.8k forks

Report repository

thaJeztah Merge pull request #4453 from thaJezt... 17 hours ago 8,901

.circleci	dummy circleci config	last year
.github	update go to go1.20.6	last week
cli-plugins	cli: SetupRootCommand: remove redundan...	last month
cli	cli/command/container: waitExitOrRemove...	yesterday
cmd/docker	cli: SetupRootCommand: remove redundan...	last month

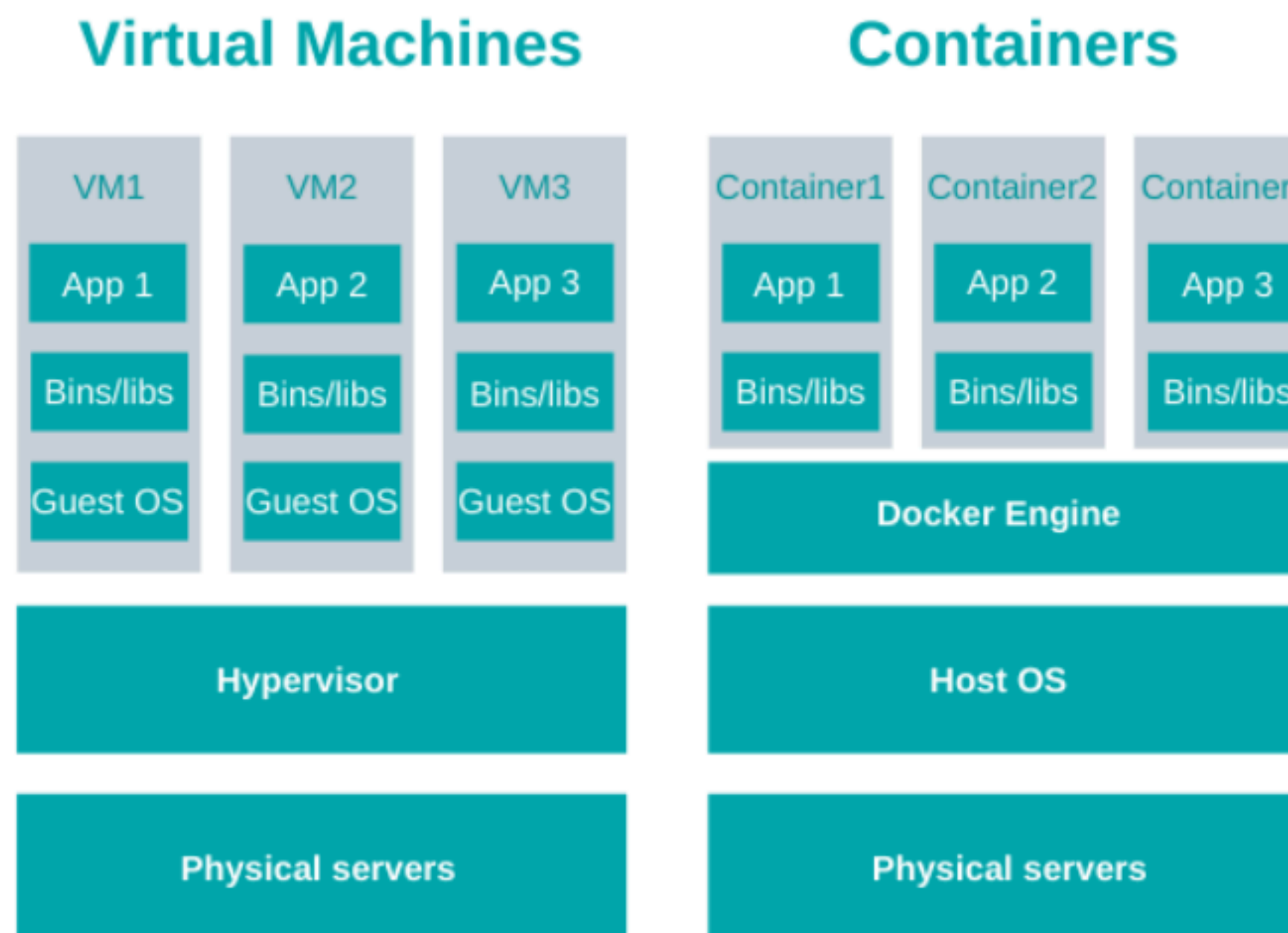
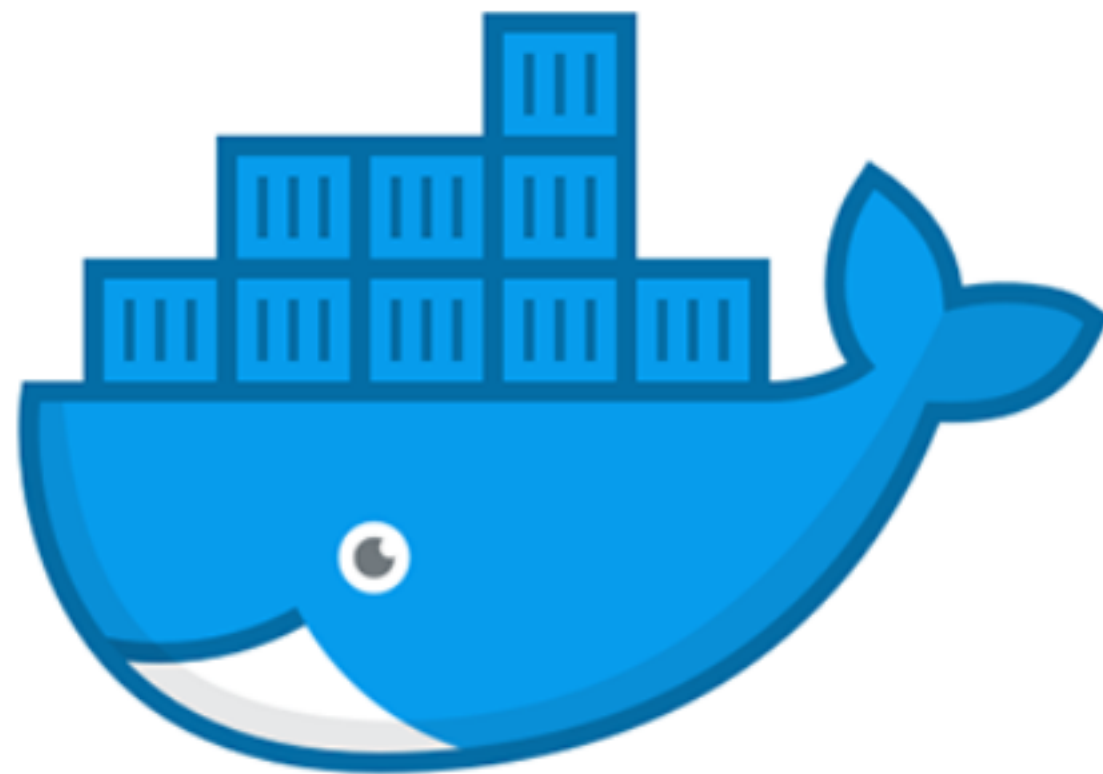
Docker Basics

What is docker?

Docker is a tool designed to make it easier to create, deploy, and run applications by using containers.

Why docker?

Docker is useful to automate the deployment of applications inside a software containers, which makes the applications easy to ship and run virtually anywhere (i.e, platform independent). The Docker container processes run on the host kernel, unlike VM which runs processes in guest kernel.



Docker Basics

Build

Build an image from the Dockerfile in the current directory and tag the image

```
docker build -t myimage:1.0 .
```

List all images that are locally stored with the Docker Engine

```
docker image ls
```

Delete an image from the local image store

```
docker image rm alpine:3.4
```

Share

Pull an image from a registry

```
docker pull myimage:1.0
```

Retag a local image with a new image name and tag

```
docker tag myimage:1.0 myrepo/  
myimage:2.0
```

Push an image to a registry

```
docker push myrepo/myimage:2.0
```

Run

Run a container from the Alpine version 3.9 image, name the running container "web" and expose port 5000 externally, mapped to port 80 inside the container.

```
docker container run --name web -p  
5000:80 alpine:3.9
```

Stop a running container through SIGTERM

```
docker container stop web
```

Stop a running container through SIGKILL

```
docker container kill web
```

List the networks

```
docker network ls
```

List the running containers (add `--all` to include stopped containers)

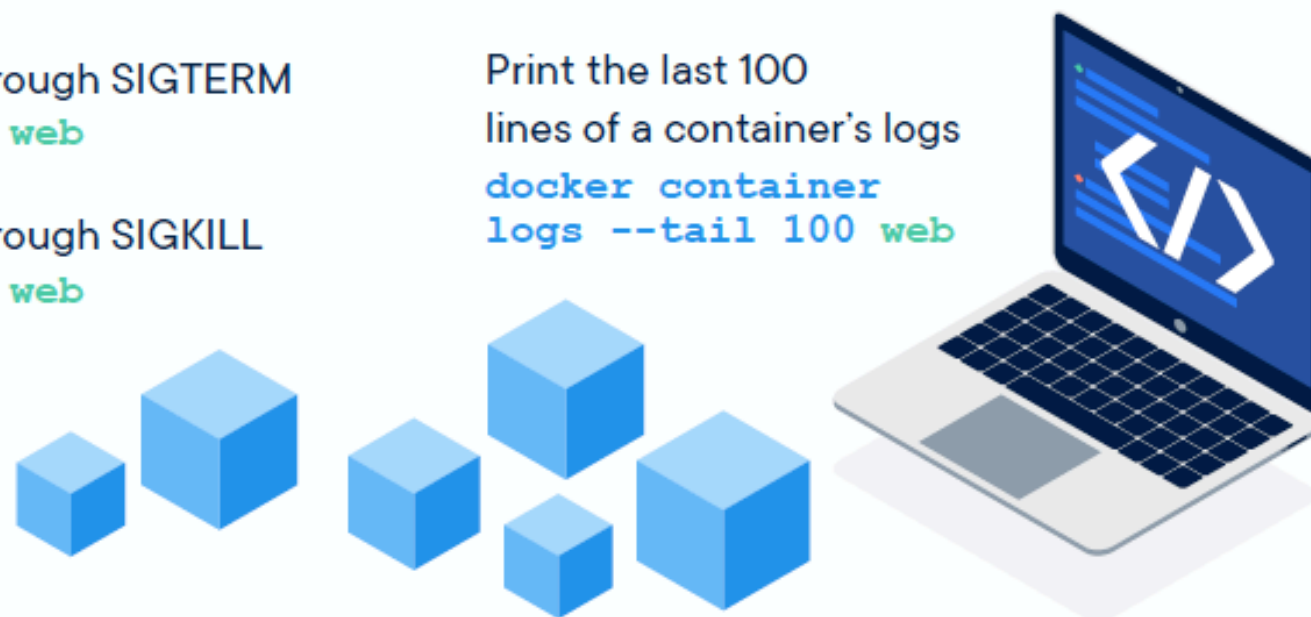
```
docker container ls
```

Delete all running and stopped containers

```
docker container rm -f $(docker ps -aq)
```

Print the last 100 lines of a container's logs

```
docker container  
logs --tail 100 web
```



```
FROM ubuntu
```

```
CMD echo "Hello world"
```

```
FROM openjdk
```

```
COPY target/hello.jar /usr/src/hello.jar
```

```
CMD java -cp /usr/src/hello.jar org.example.App
```


Docker Keywords

Dockerfile reference

Usage

Format

Parser directives

escape

Environment replacement

.dockerignore file

FROM

RUN

Known issues [RUN]

CMD

LABEL

MAINTAINER [deprecated]

EXPOSE

ENV

ADD

COPY

ENTRYPOINT

Exec form ENTRYPOINT example

Shell form ENTRYPOINT example

Understand how CMD and
ENTRYPOINT interact

VOLUME

USER

WORKDIR

ARG

Impact on build caching

ONBUILD

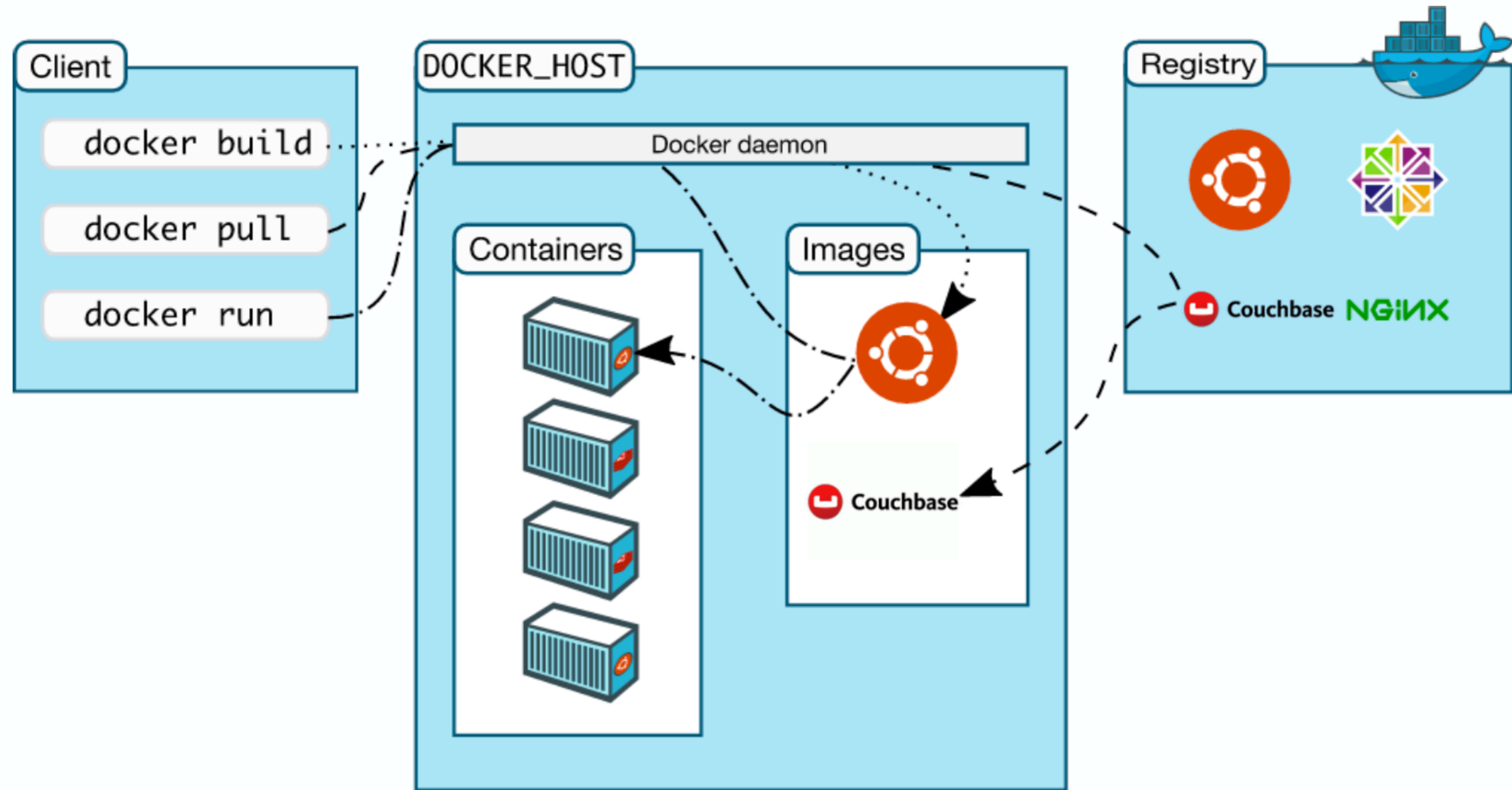
STOPSIGNAL

HEALTHCHECK

SHELL

Dockerfile examples

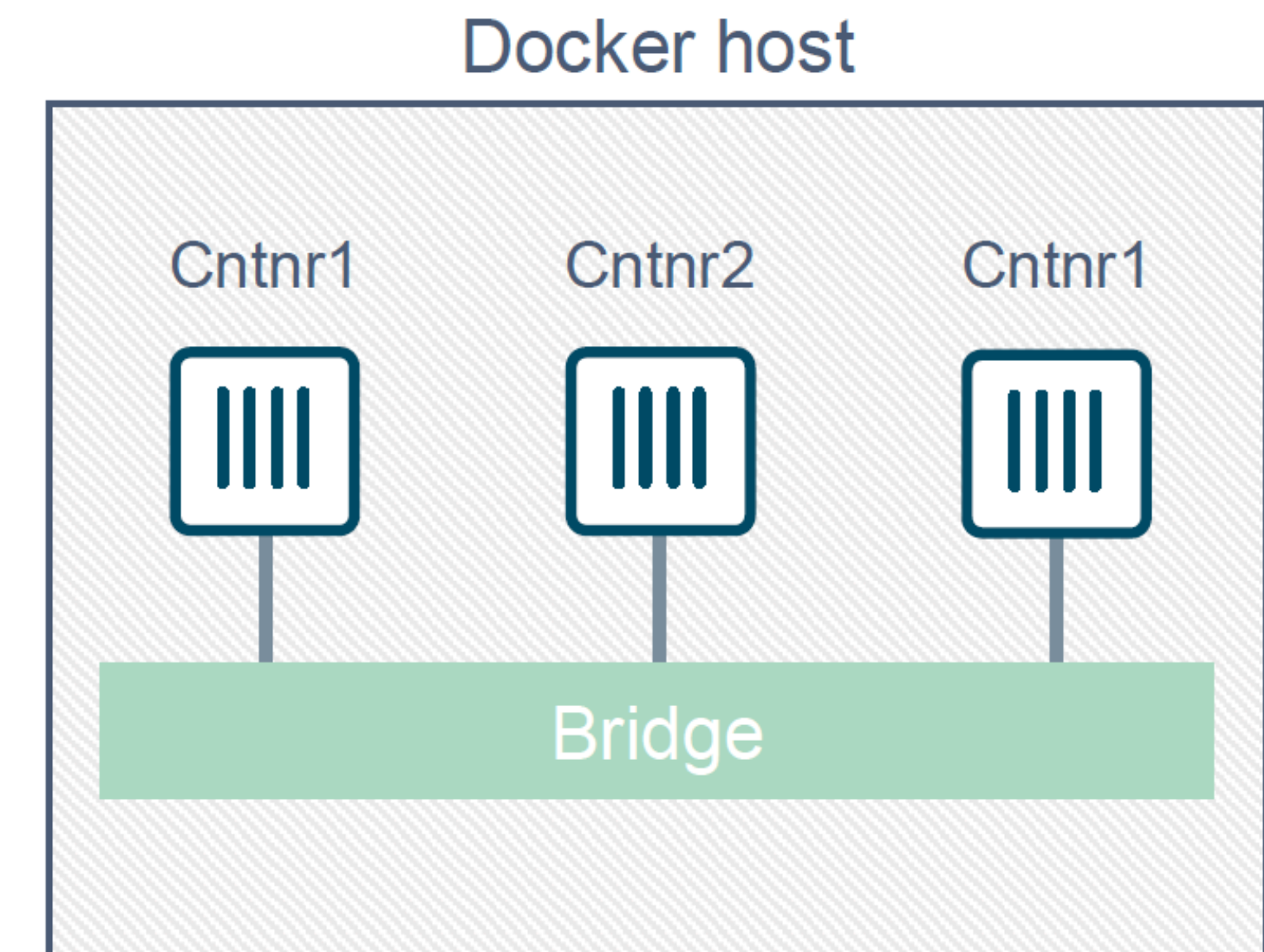
Docker WorkFlows



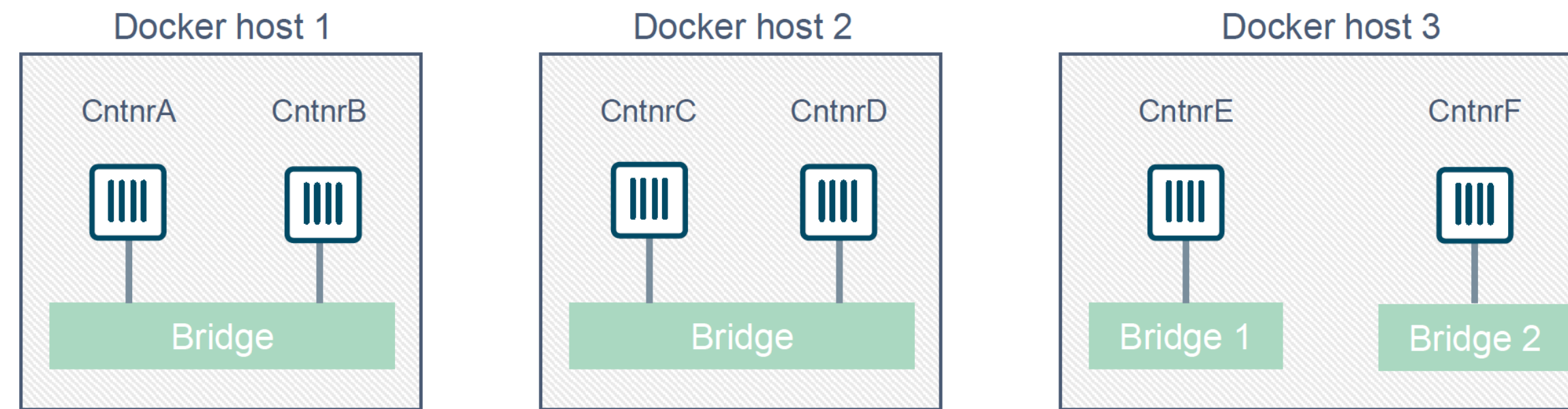
Docker Networking

What is Docker Bridge Networking?

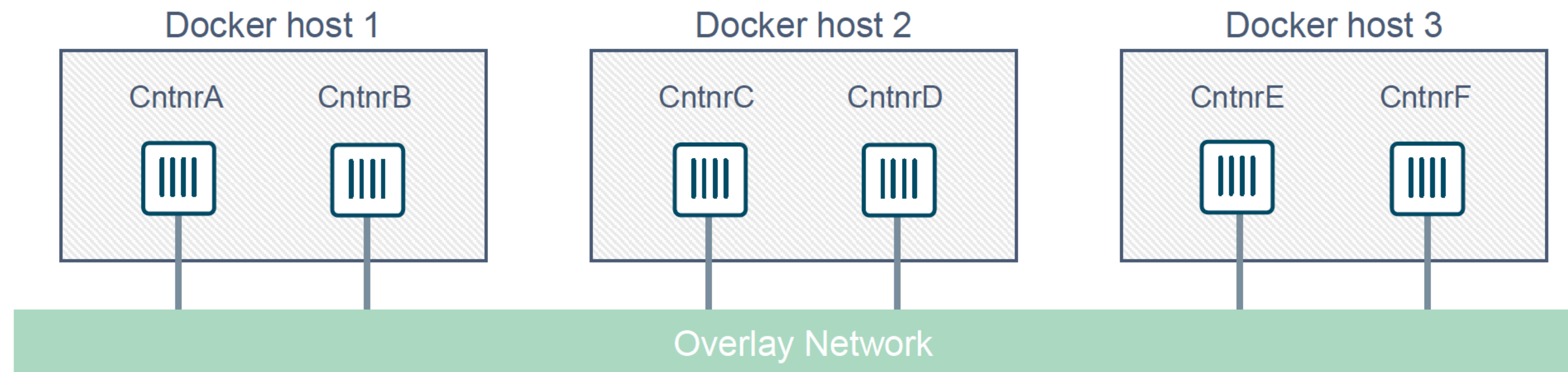
- The bridge driver creates a bridge (virtual switch) on a single Docker host
- Containers get plumbed into this bridge
- All containers on this bridge can communicate
- The bridge is a private network restricted to a single Docker host



Docker Networking



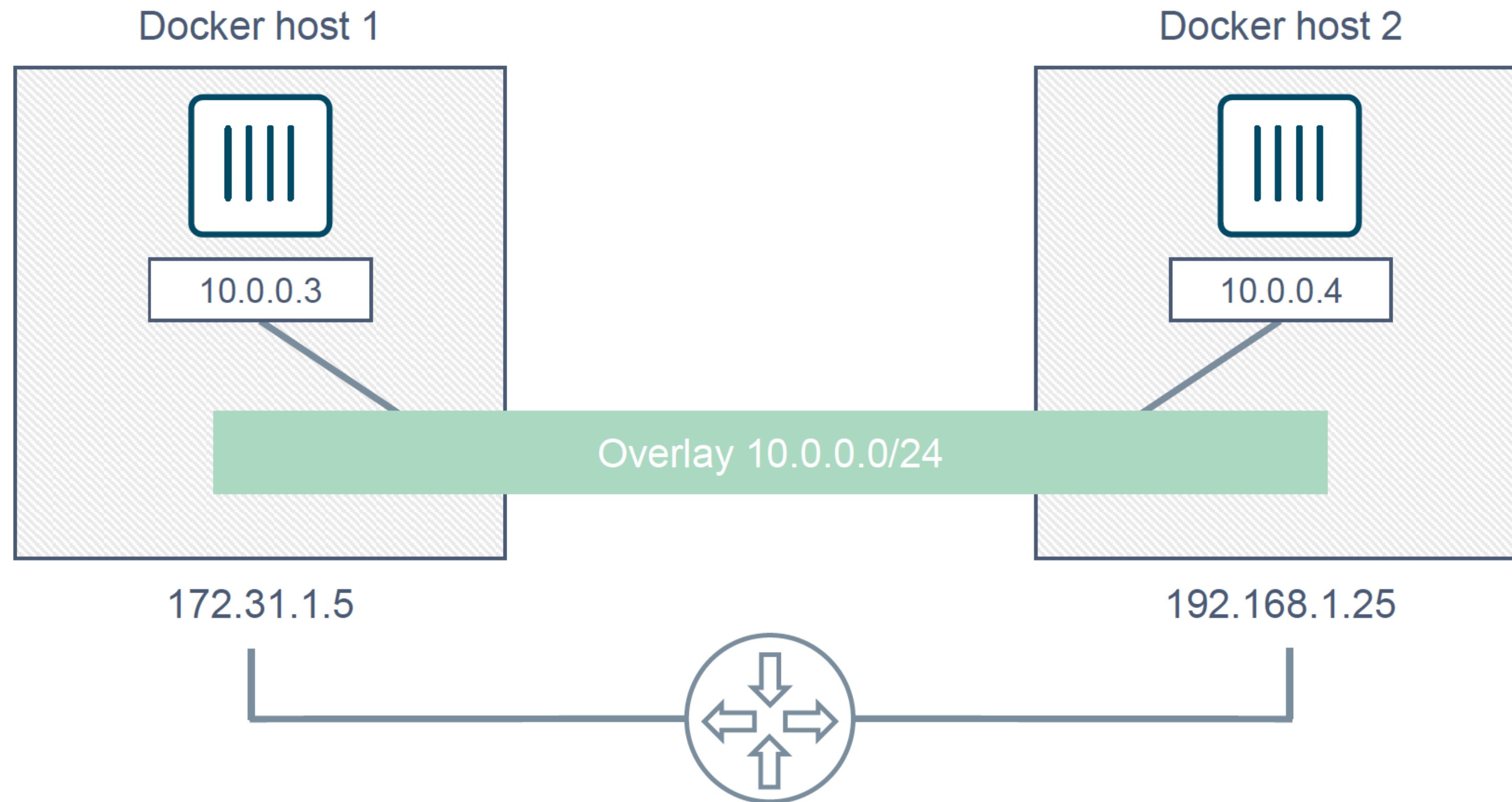
Containers on different **bridge** networks cannot communicate



All containers on the **overlay** network can communicate!

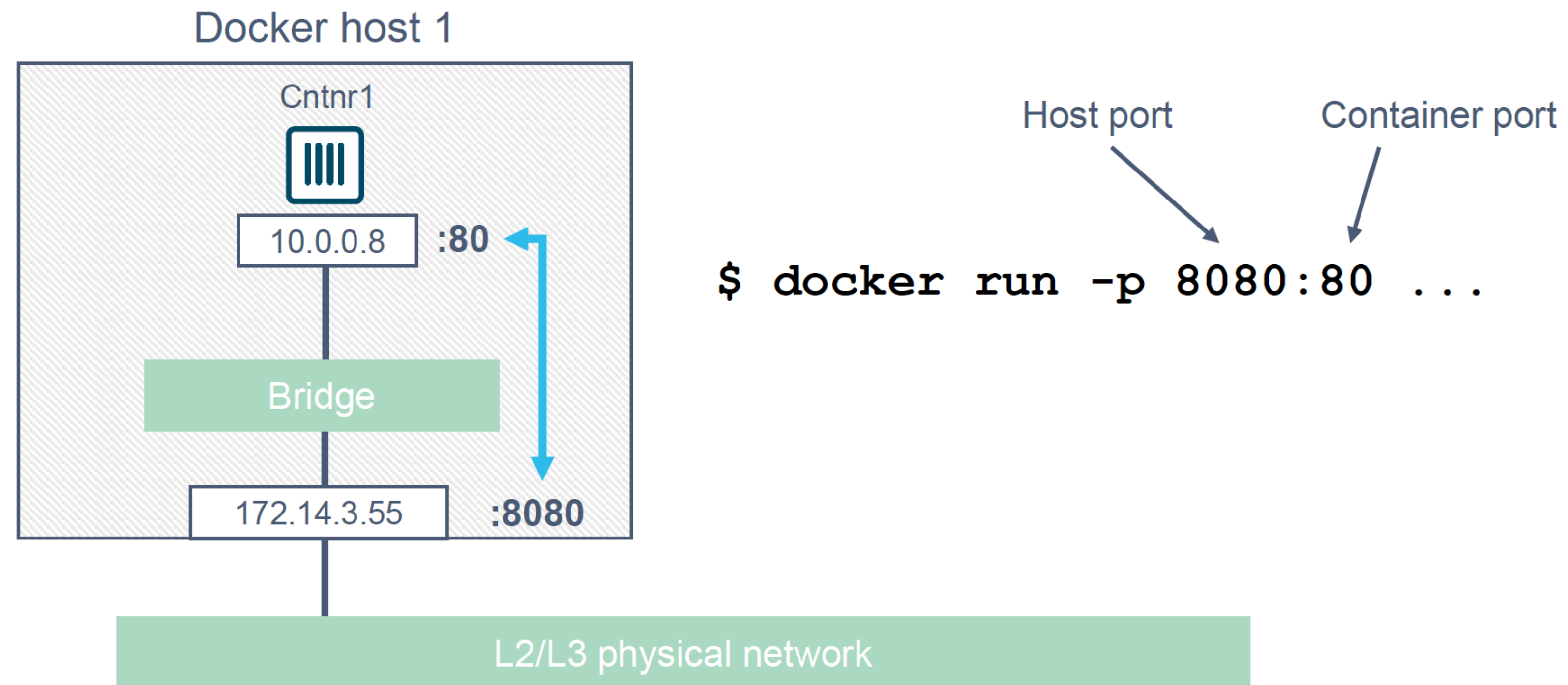
Docker Networking

Building an Overlay Network (High level)



Docker Networking

Docker Bridge Networking and Port Mapping



Docker Compose

- Defining and running multi-container applications
- Configuration defined in one or more files
 - `docker-compose.yml` (default)
 - `docker-compose.override.yml` (default)
 - Multiple files specified using `-f`
- Deployed as Docker Stack
- Great for dev, staging, and CI

Docker Compose (1 service)

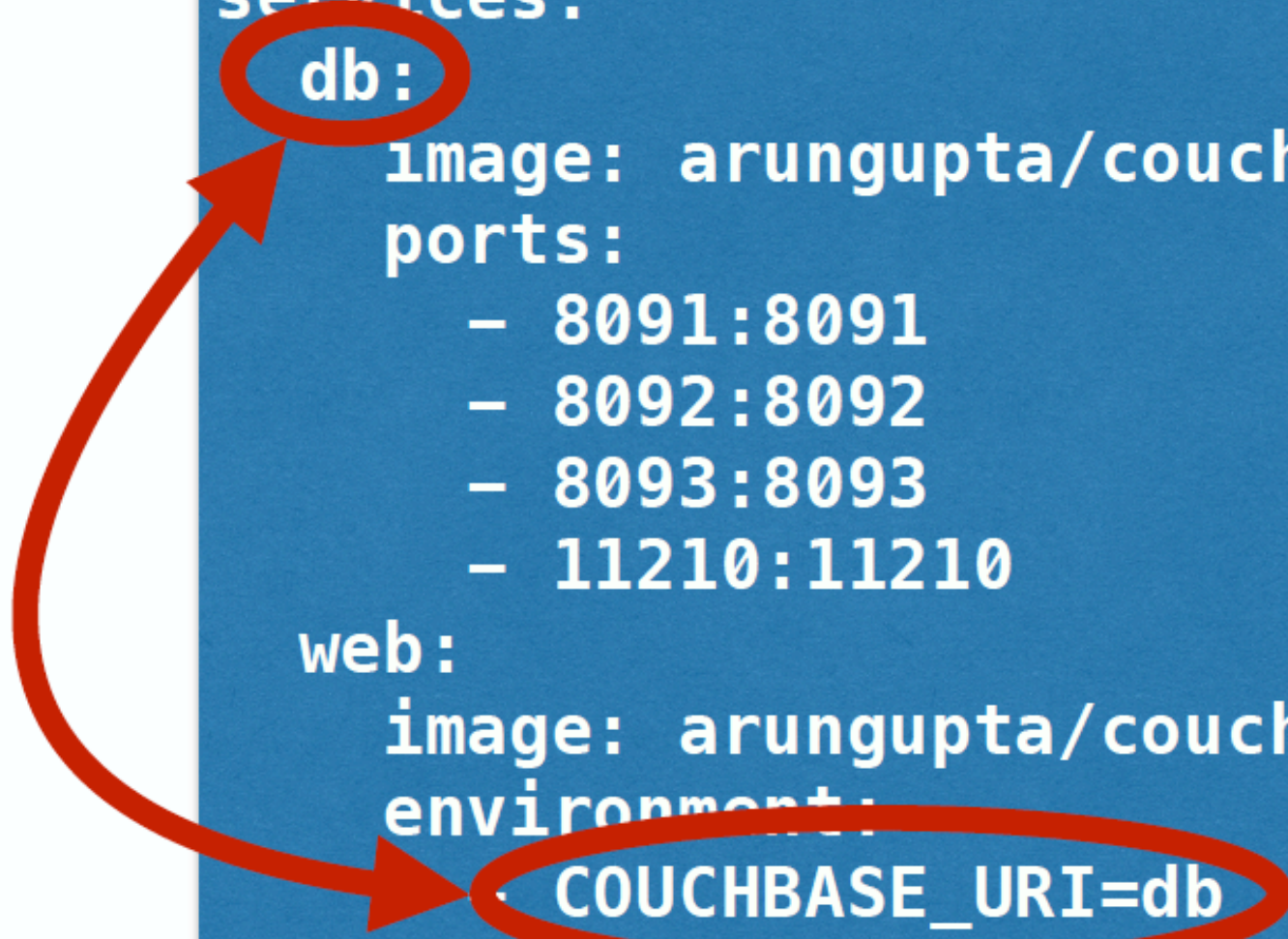
```
version: "3"
services:
  db:
    image: couchbase
    volumes:
      - ~/couchbase:/opt/couchbase/var
    ports:
      - 8091:8091
      - 8092:8092
      - 8093:8093
      - 11210:11210
```

```
docker-compose up -d
```

```
docker stack deploy \
--compose-file=docker-compose.yml \
couchbase
```

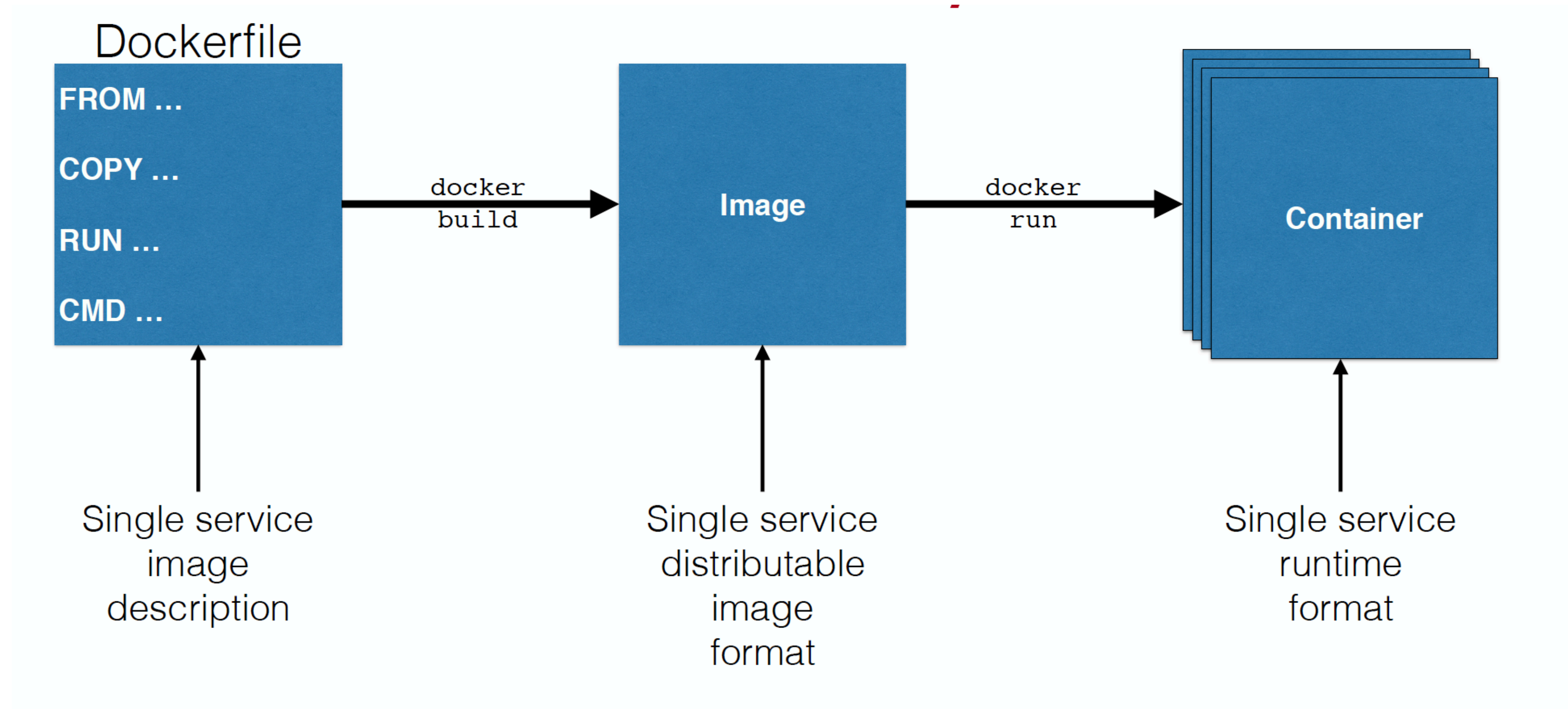

Docker Compose (2 service)

```
version: "3"
services:
  db:
    image: arungupta/couchbase:travel
    ports:
      - 8091:8091
      - 8092:8092
      - 8093:8093
      - 11210:11210
  web:
    image: arungupta/couchbase-wildfly-javaee:travel
    environment:
      - COUCHBASE_URI=db
    ports:
      - 8080:8080
      - 9990:9990
```

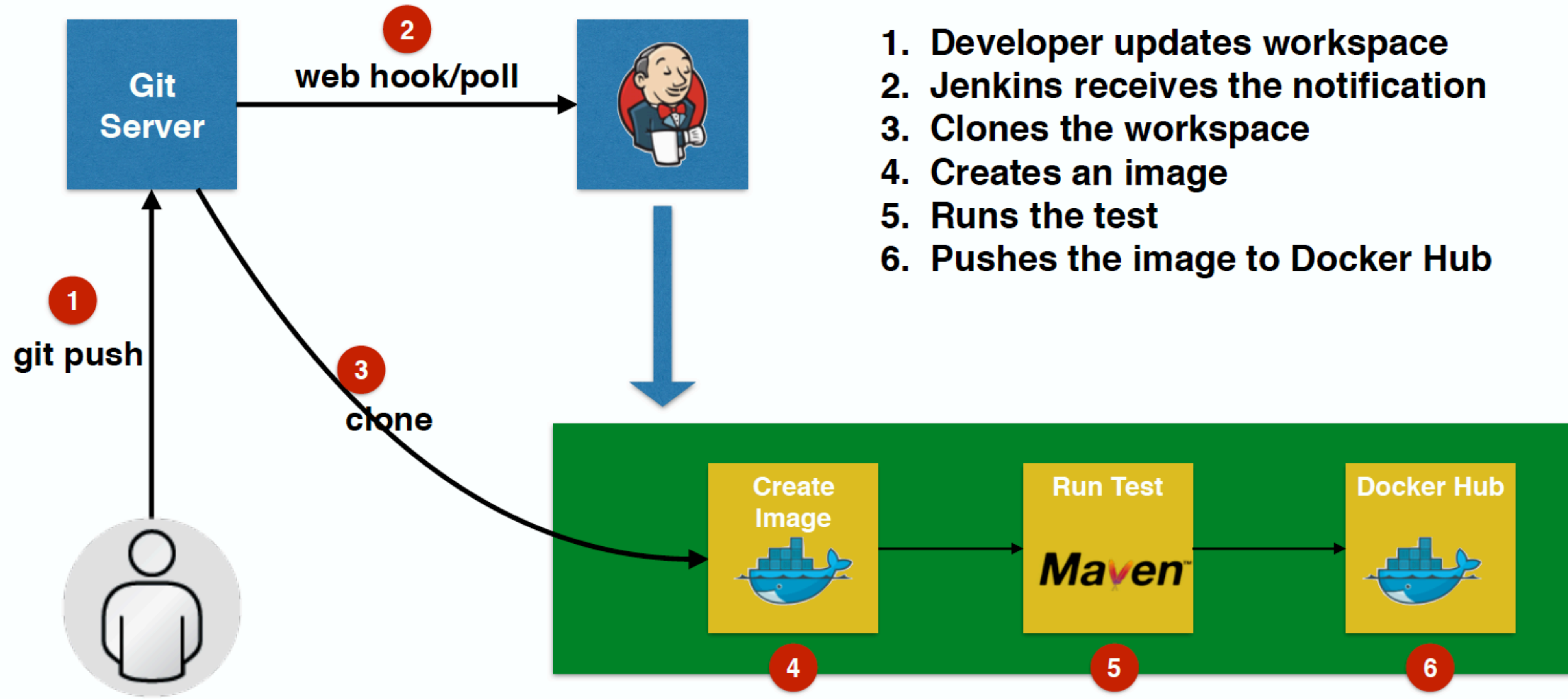


```
docker stack deploy \
--compose-file=docker-compose.yml \
couchbase
```


Docker LifeCycle



CI/CD with Docker + Jenkins



Docker Compose

```
1 ► docker network create mongo-net
2
3 docker run -d \
4   --network mongo-net \
5   --name mongodb \
6   -p 27017:27017 \
7   -v mongodb-data:/data/db \
8   -e MONGO_INITDB_ROOT_USERNAME=root \
9   -e MONGO_INITDB_ROOT_PASSWORD=admin \
10  mongo
11
12 docker run -d \
13   --network mongo-net \
14   --name mongo-express \
15   -p 8081:8081 \
16   -e ME_CONFIG_MONGODB_ADMINUSERNAME="root" \
17   -e ME_CONFIG_MONGODB_ADMINPASSWORD="admin" \
18   -e ME_CONFIG_MONGODB_SERVER="mongodb" \
19   -e ME_CONFIG_BASICAUTH_USERNAME="root" \
20   -e ME_CONFIG_BASICAUTH_PASSWORD="admin" \
21  mongo-express
22
```

```
1  version: '3.3'
2  ► services:
3  ►   mongo:
4     image: mongo:latest
5     ports:
6       - "27017:27017"
7     restart: always
8     volumes:
9       - ./mongodb-data:/data/db
10    environment:
11      MONGO_INITDB_ROOT_USERNAME: rootuser
12      MONGO_INITDB_ROOT_PASSWORD: rootpass
13
14  ►   mongo-express:
15     image: mongo-express:latest
16     restart: always
17     ports:
18       - "8081:8081"
19     environment:
20      ME_CONFIG_MONGODB_ADMINUSERNAME: rootuser
21      ME_CONFIG_MONGODB_ADMINPASSWORD: rootpass
22      ME_CONFIG_MONGODB_SERVER: mongo
23      #ME_CONFIG_MONGODB_PORT: "27017"
24      #ME_CONFIG_MONGODB_URL: "mongodb://rootuser:rootpass@m
```


Kubernetes

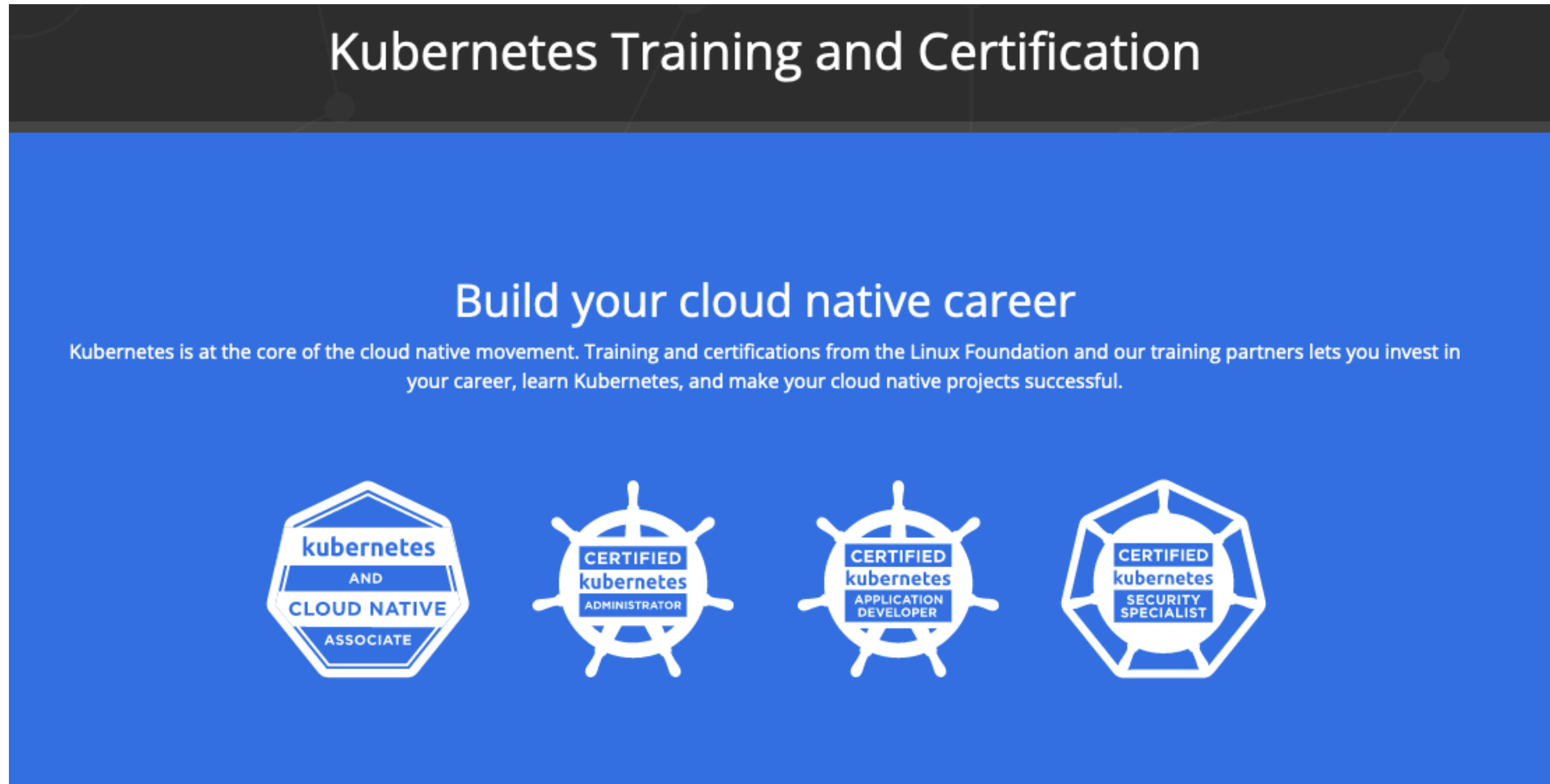


WorkShop Goals

- Revisit trends in app design and deployment
- Get (more) familiar with containers
- Build a Docker container image for a demo Flask app
- Learn about Kubernetes architecture and objects
- Create and access a Kubernetes cluster
- Deploy Flask app to Kubernetes cluster
- By the end, have a load-balanced Flask app

Definition

[Kubernetes](#), also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.



Kubernetes Training and Certification

Build your cloud native career

Kubernetes is at the core of the cloud native movement. Training and certifications from the Linux Foundation and our training partners lets you invest in your career, learn Kubernetes, and make your cloud native projects successful.



Latest version

- [v1.23](#) (this documentation)

Older versions

- [v1.22](#)
- [v1.21](#)
- [v1.20](#)
- [v1.19](#)

Automated rollouts and rollbacks

Kubernetes progressively rolls out changes to your application or its configuration, while monitoring application health to ensure it doesn't kill all your instances at the same time. If something goes wrong, Kubernetes will rollback the change for you. Take advantage of a growing ecosystem of deployment solutions.

Storage orchestration

Automatically mount the storage system of your choice, whether from local storage, a public cloud provider such as [GCP](#) or [AWS](#), or a network storage system such as NFS, iSCSI, Gluster, Ceph, Cinder, or Flocker.

Automatic bin packing

Automatically places containers based on their resource requirements and other constraints, while not sacrificing availability. Mix critical and best-effort workloads in order to drive up utilization and save even more resources.

IPv4/IPv6 dual-stack

Allocation of IPv4 and IPv6 addresses to Pods and Services

Self-healing

Restarts containers that fail, replaces and reschedules containers when nodes die, kills containers that don't respond to your user-defined health check, and doesn't advertise them to clients until they are ready to serve.

Service discovery and load balancing

No need to modify your application to use an unfamiliar service discovery mechanism. Kubernetes gives Pods their own IP addresses and a single DNS name for a set of Pods, and can load-balance across them.

Secret and configuration management

Deploy and update secrets and application configuration without rebuilding your image and without exposing secrets in your stack configuration.

Batch execution

In addition to services, Kubernetes can manage your batch and CI workloads, replacing containers that fail, if desired.

Horizontal scaling

Scale your application up and down with a simple command, with a UI, or automatically based on CPU usage.

Designed for extensibility

Add features to your Kubernetes cluster without changing upstream source code.

API Reference

- [Glossary](#) - a comprehensive, standardized list of Kubernetes terminology
- [Kubernetes API Reference](#)
- [One-page API Reference for Kubernetes v1.23](#)
- [Using The Kubernetes API](#) - overview of the API for Kubernetes.
- [API access control](#) - details on how Kubernetes controls API access
- [Well-Known Labels, Annotations and Taints](#)

CLI

- [kubectl](#) - Main CLI tool for running commands and managing Kubernetes clusters.
 - [JSONPath](#) - Syntax guide for using [JSONPath expressions](#) with kubectl.
- [kubeadm](#) - CLI tool to easily provision a secure Kubernetes cluster.

Config APIs

This section hosts the documentation for "unpublished" APIs which are used to configure kubernetes components or tools. Most of these APIs are not exposed by the API server in a RESTful way though they are essential for a user or an operator to use or manage a cluster.

- [kube-apiserver configuration \(v1\)](#)
- [kubelet configuration \(v1alpha1\)](#) and [kubelet configuration \(v1beta1\)](#)
- [kube-scheduler configuration \(v1beta2\)](#) and [kube-scheduler configuration \(v1beta3\)](#)
- [kube-proxy configuration \(v1alpha1\)](#)
- [audit.k8s.io/v1 API](#)
- [Client authentication API \(v1beta1\)](#) and [Client authentication API \(v1\)](#)
- [WebhookAdmission configuration \(v1\)](#)

Officially supported client libraries

To call the Kubernetes API from a programming language, you can use [client libraries](#). Officially supported client libraries:

- [Kubernetes Go client library](#)
- [Kubernetes Python client library](#)
- [Kubernetes Java client library](#)
- [Kubernetes JavaScript client library](#)
- [Kubernetes C# client library](#)
- [Kubernetes Haskell client library](#)

Components

- [kubelet](#) - The primary agent that runs on each node. The kubelet takes a set of PodSpecs and ensures that the described containers are running and healthy.
- [kube-apiserver](#) - REST API that validates and configures data for API objects such as pods, services, replication controllers.
- [kube-controller-manager](#) - Daemon that embeds the core control loops shipped with Kubernetes.
- [kube-proxy](#) - Can do simple TCP/UDP stream forwarding or round-robin TCP/UDP forwarding across a set of back-ends.
- [kube-scheduler](#) - Scheduler that manages availability, performance, and capacity.
 - [Scheduler Policies](#)
 - [Scheduler Profiles](#)
- List of [ports and protocols](#) that should be open on control plane and worker nodes

Declarative vs. Imperative

```
on master +1 !5 ?1 > RAM 6.65G kubectl get pods -w --namespace helm-mysql
```

```
apiVersion: v1
kind: Pod
metadata:
  name: flask-pod
  labels:
    app: flask-helloworld
spec:
  containers:
  - name: flask
    image: worldbosskafka/flask-helloworld:latest
    ports:
    - containerPort: 5000
```

Getting Started with
Containers and Kubernetes

App Modernization & Microservices

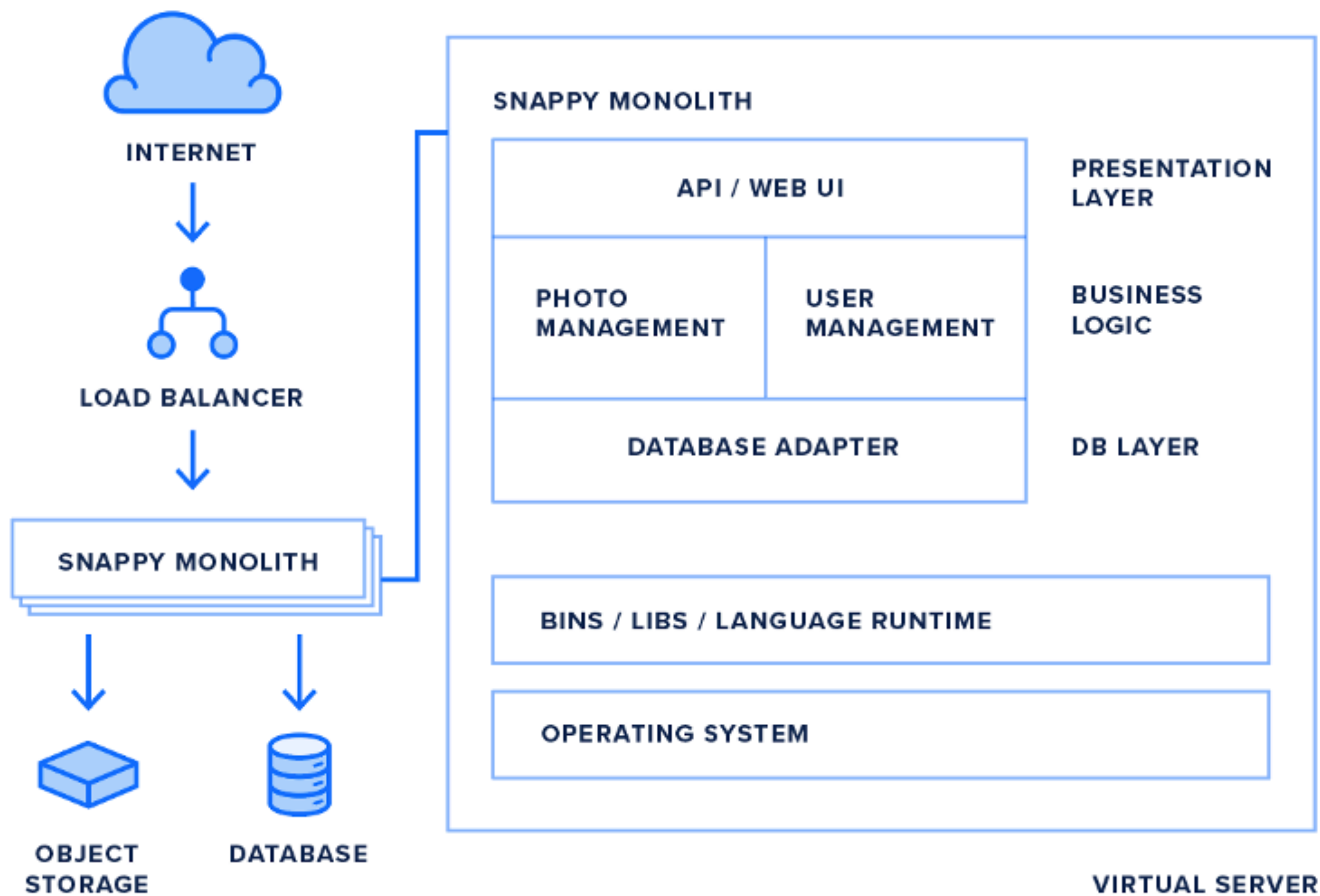
Revisiting Containers

Container Clusters

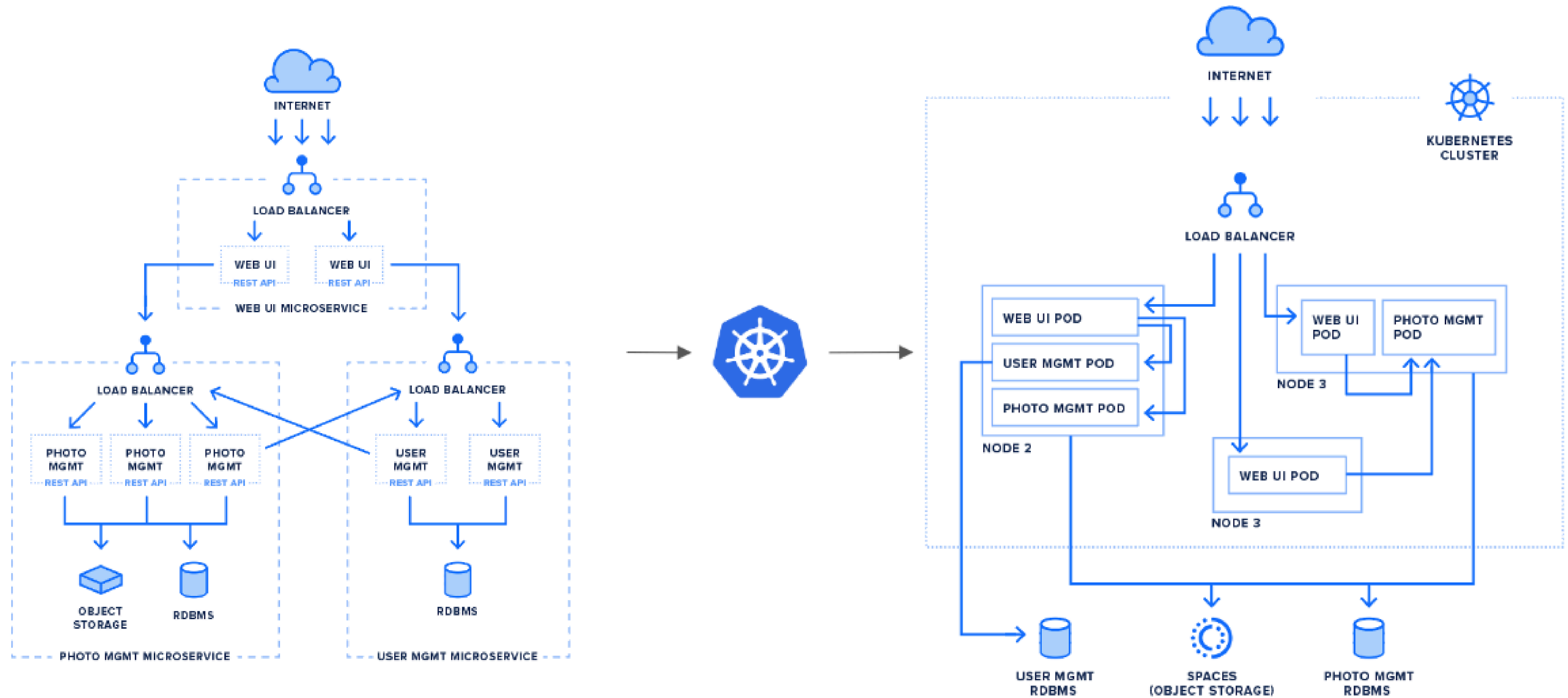
App Modernization & Microservices

Kubernetes Objects: Pods and Workloads

The Monolith



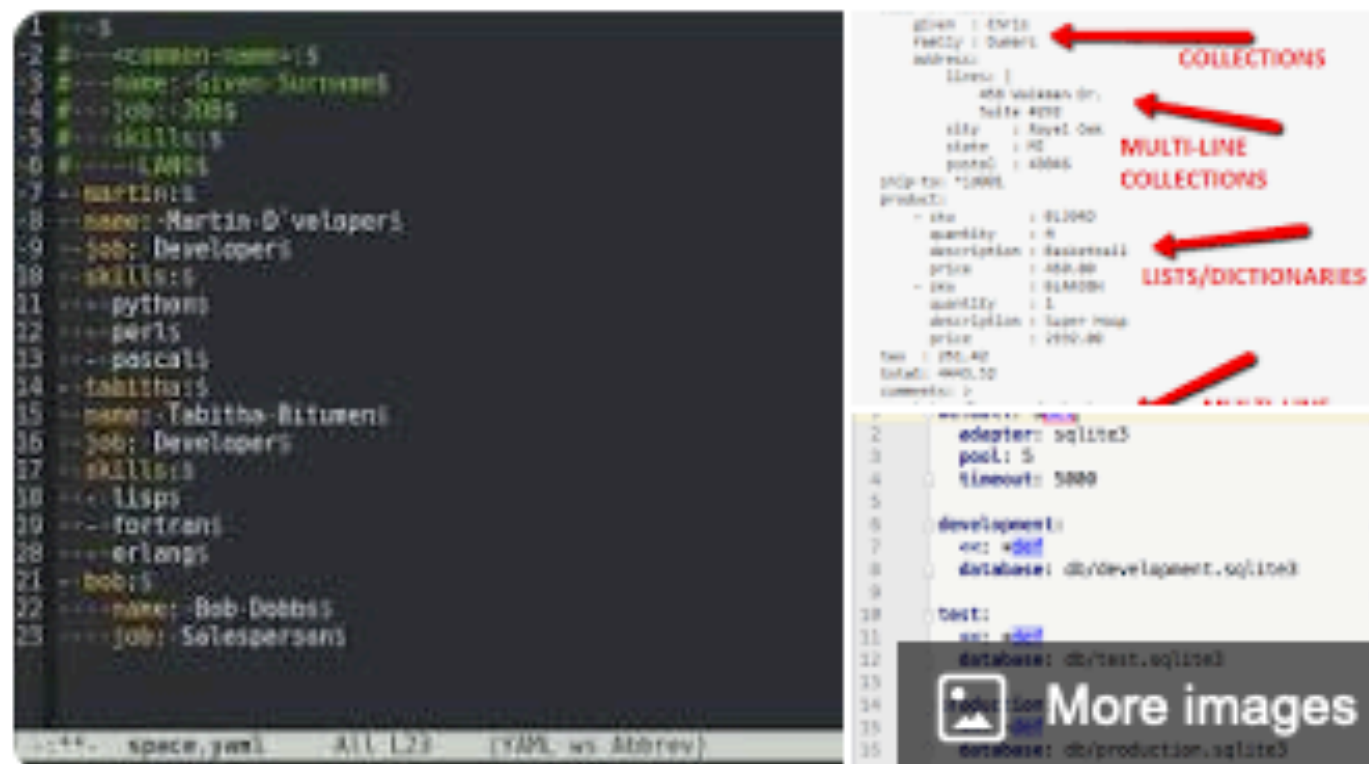
Breaking the Monolith



Prerequisite

YAML

Programming language



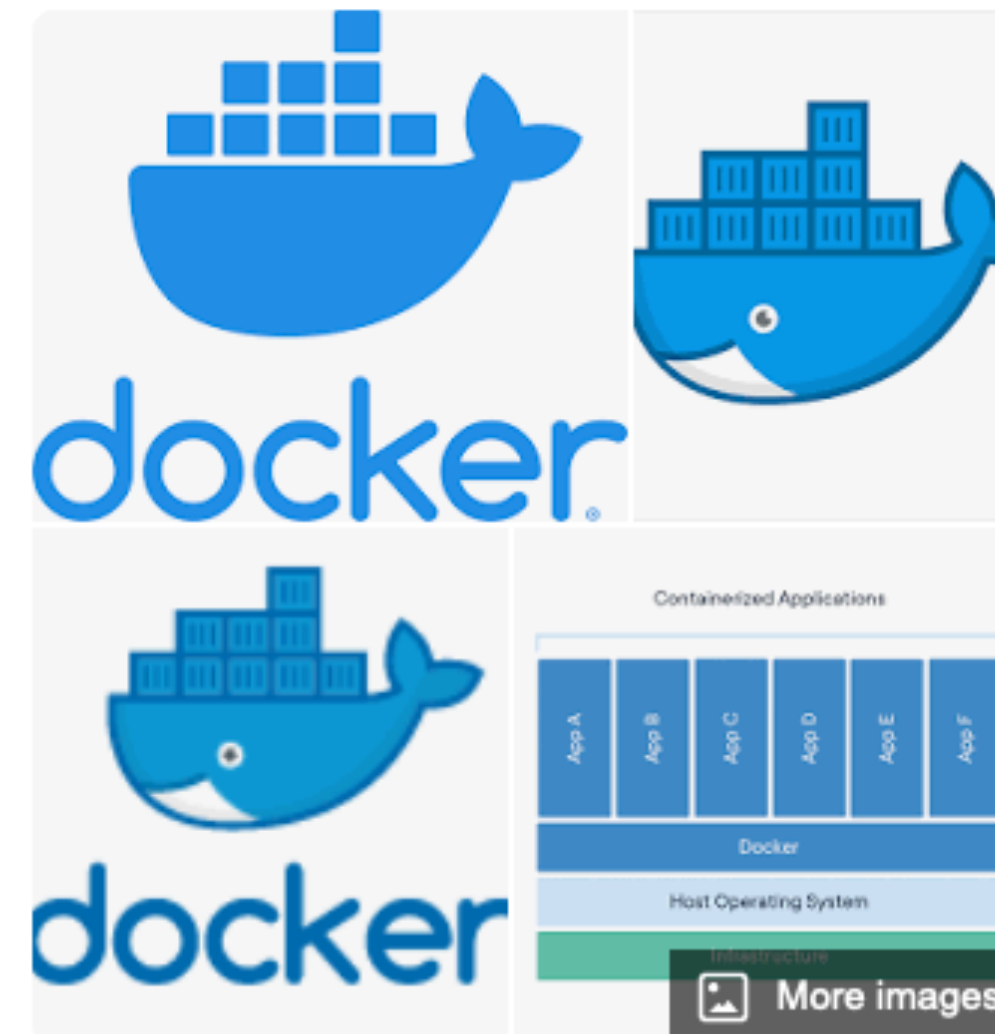
YAML is a human-readable data-serialization language. It is commonly used for configuration files and in applications where data is being stored or transmitted. YAML targets many of the same communications applications as Extensible Markup Language but has a minimal syntax which intentionally differs from SGML. [Wikipedia](#)

Initial release: 11 May 2001; 20 years ago

Latest release: 1.2 (Third Edition); 1 October 2009; 12 years ago

Docker

Software



Docker is a set of platform as a service products that use OS-level virtualization to deliver software in packages called containers. Containers are isolated from one another and bundle their own software, libraries and configuration files; they can communicate with each other through well-defined channels. [Wikipedia](#)

Developer(s): Docker, Inc.

License: Free / Paid

Repository: github.com/moby/moby

Platform: x86-64, ARM, s390x, ppc64le

Written in: Go

Original author(s): Solomon Hykes

Operating system: Linux, Windows, macOS

Go

Programming language



Go is a statically typed, compiled programming language designed at Google by Robert Griesemer, Rob Pike, and Ken Thompson. Go is syntactically similar to C, but with memory safety, garbage collection, structural typing, and CSP-style concurrency. [Wikipedia](#)

Implementation language: Go, Assembly language (gc); C++ (gofrontend)

Typing discipline: Inferred, static, strong, structural, nominal

First appeared: November 10, 2009; 12 years ago

OS: DragonFly BSD, FreeBSD, Linux, macOS, NetBSD, OpenBSD, Plan 9, Solaris, Windows

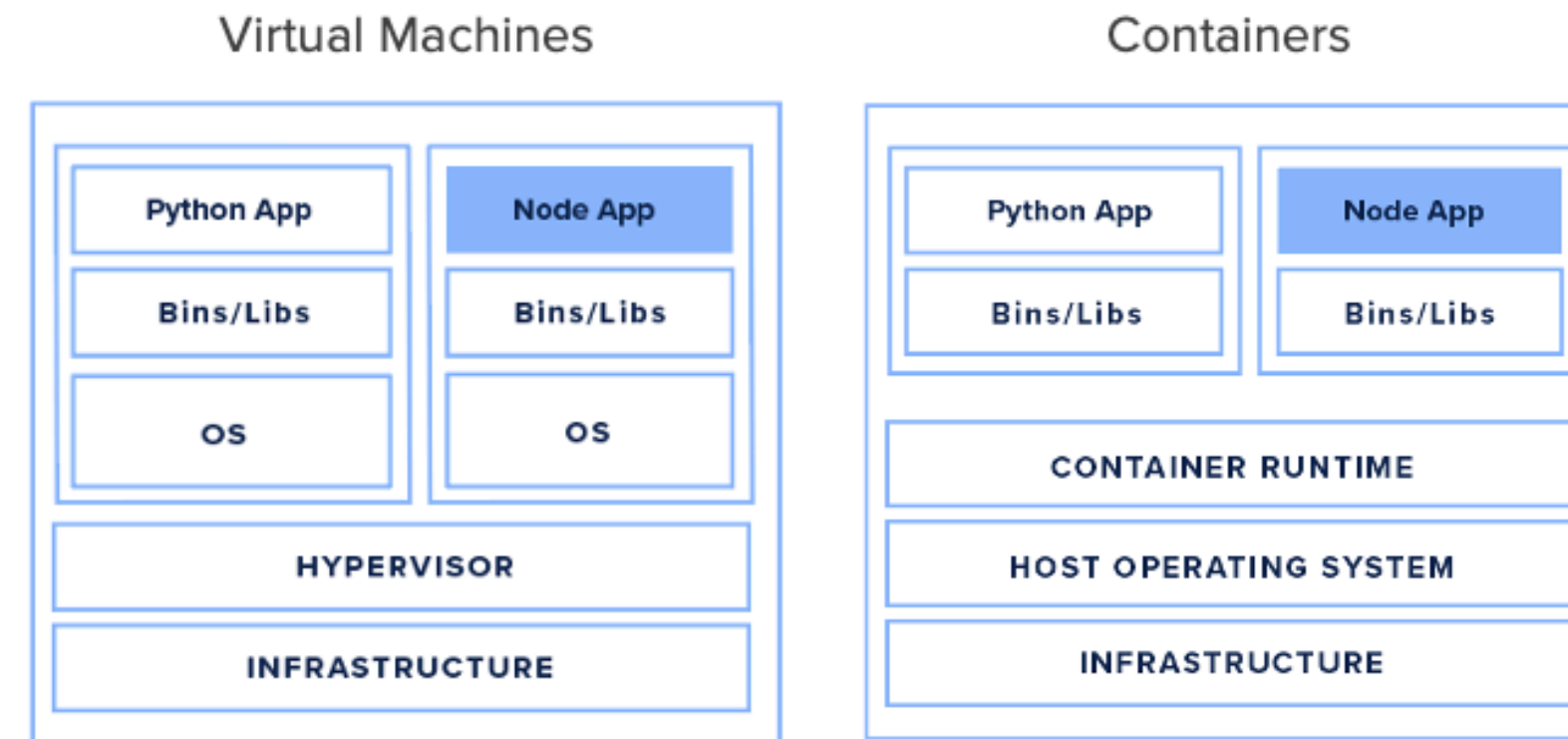
License: 3-clause BSD + patent grant

Paradigm: Multi-paradigm: concurrent imperative, object-oriented

Designed by: Robert Griesemer, Rob Pike, Ken Thompson

What is a Container?

- VMs vs. Containers
- Container features
 - Lightweight
 - Portable
 - Isolated

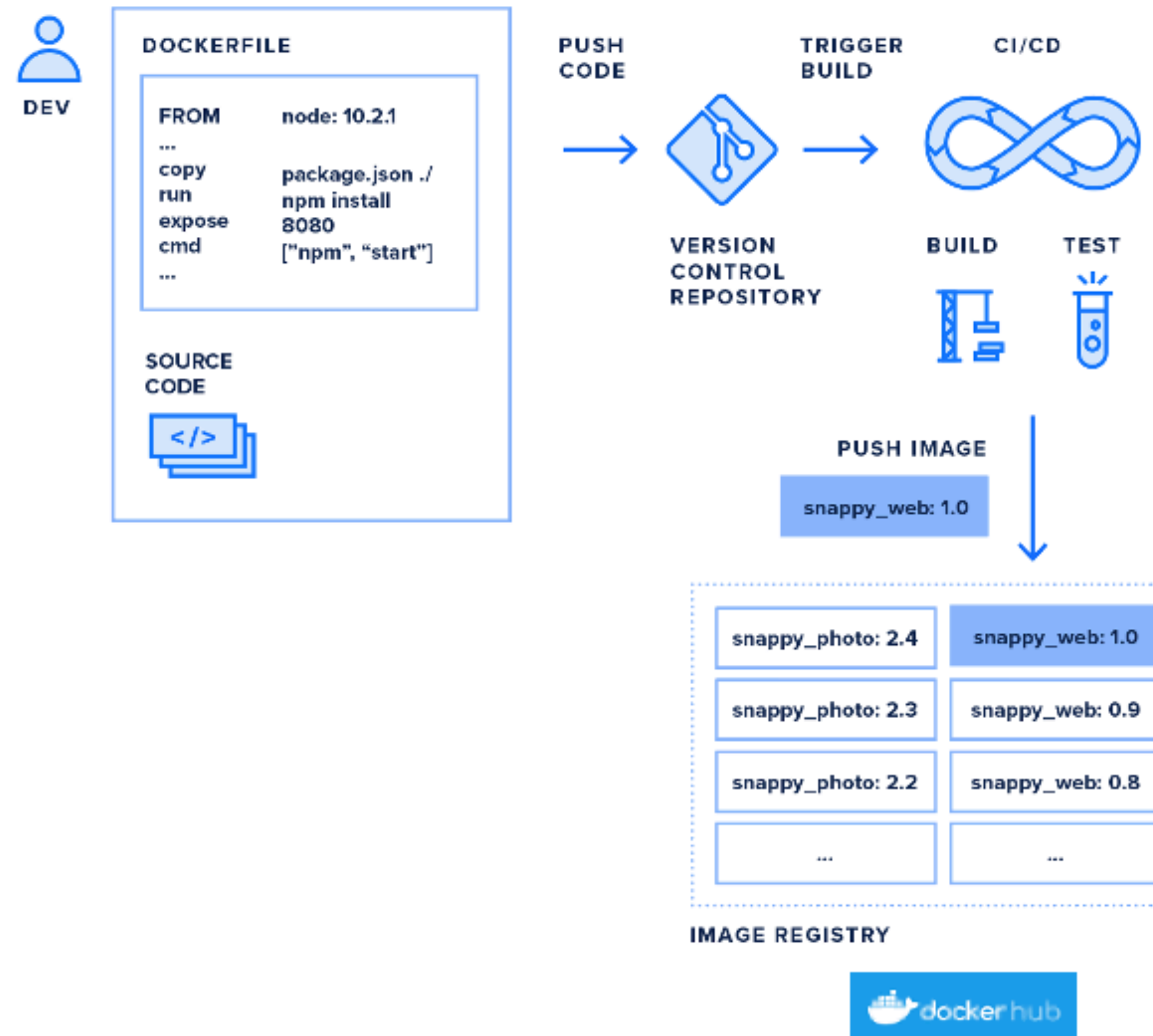


But what are they, really?

- A package of application code and all of its dependencies
 - Includes everything needed to run the application
- Built around two Linux kernel features
 - Namespaces: Process isolation
 - Cgroups: Resource limits
- Why are they important?

Container Ecosystem

- Container Images
- Containers
- Container Runtime
- Container Registries



Minikube Tutorial

<https://www.abhishek-tiwari.com/local-development-environment-for-kubernetes-using-minikube/>

- Kubernetes.IO documentation →
- Get started with k8s w/o installation with Katacoda →
- Kubernetes Bootcamp →
- Install Kubernetes CLI kubectl →
- Create a local cluster with
 - Docker For Desktop →
 - Minikube →
 - MiniShift →
 - DinD → or Kind →

CheatSheets

<https://github.com/dennyzhang/cheatsheet-kubernetes-A4>

Containerized Flask App

App Code (cat app/app.py)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'

if __name__ == "__main__":
    app.run(debug=True, host='0.0.0.0')
```



Containerized Flask App

Dockerfile (cat app/Dockerfile)

```
FROM python:3-alpine

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

- Build & tag image
 - `docker build -t flask:v0 .`
 - `docker images`
- Run container / test
 - `docker run -p 5000:5000 flask:v0`
 - `docker ps`
 - `curl http://localhost:5000`
- Push to Docker Hub repo (optional)
- What would this look like in VM world?

Container Clusters

- What if we have 10s, 100s, 1000s of running containers on multiple VMs?
- How to deploy, scale, restart, manage all of these containers?
- What problems do they solve?
 - Management
 - Metrics
 - Health checks
 - Security
 - Abstraction of hardware
 - Networking
 - Scheduling
 - Scaling
 - Deployment
 - Rollbacks
 - Zero-downtime / blue-green
 - Service discovery

di

Brief Kubernetes History

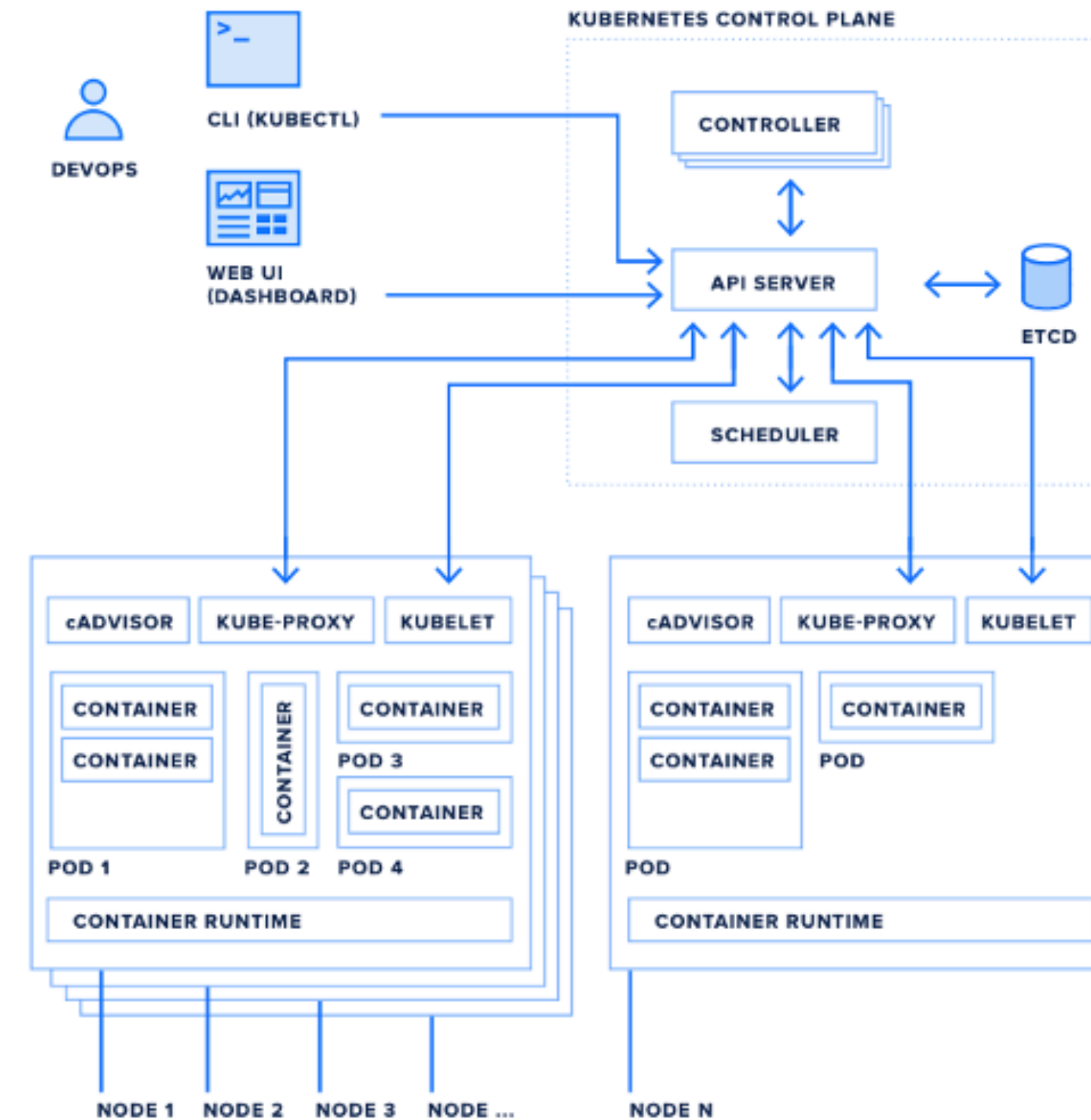


- “K8s”
- Evolved out of Borg (Google’s internal container cluster)
- Open sourced ~2014
- Grew in popularity, open source velocity increased
- Now the most popular container cluster (most cloud platforms have some sort of managed K8s offering)
- Features added regularly and frequently
- Cloud Native / CNCF - Kubernetes, Prometheus, Fluentd



Kubernetes Architecture

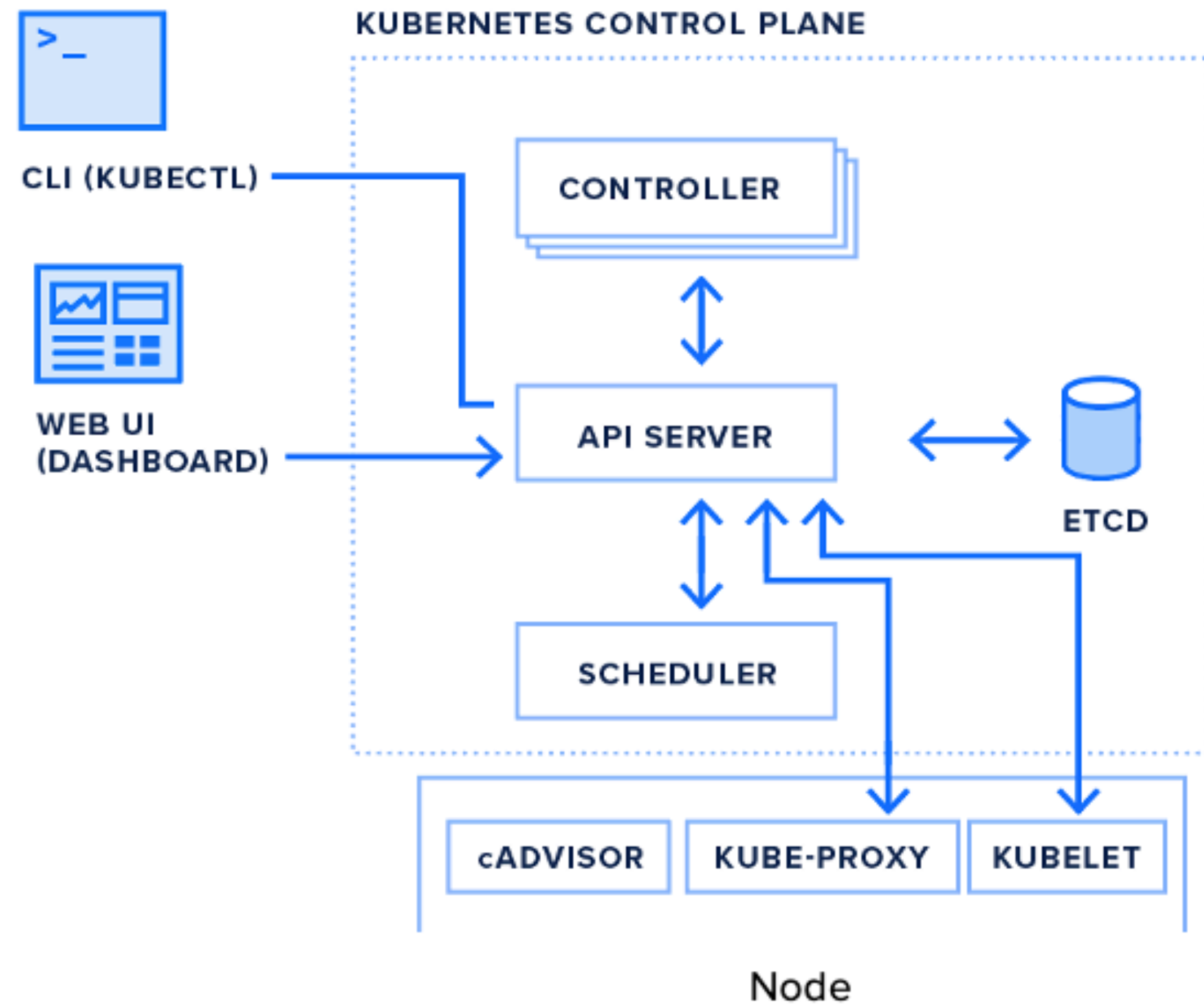
- Client-Server architecture
 - Server: Control Plane
 - Clients: Nodes



Kubernetes Architecture

- Control Plane

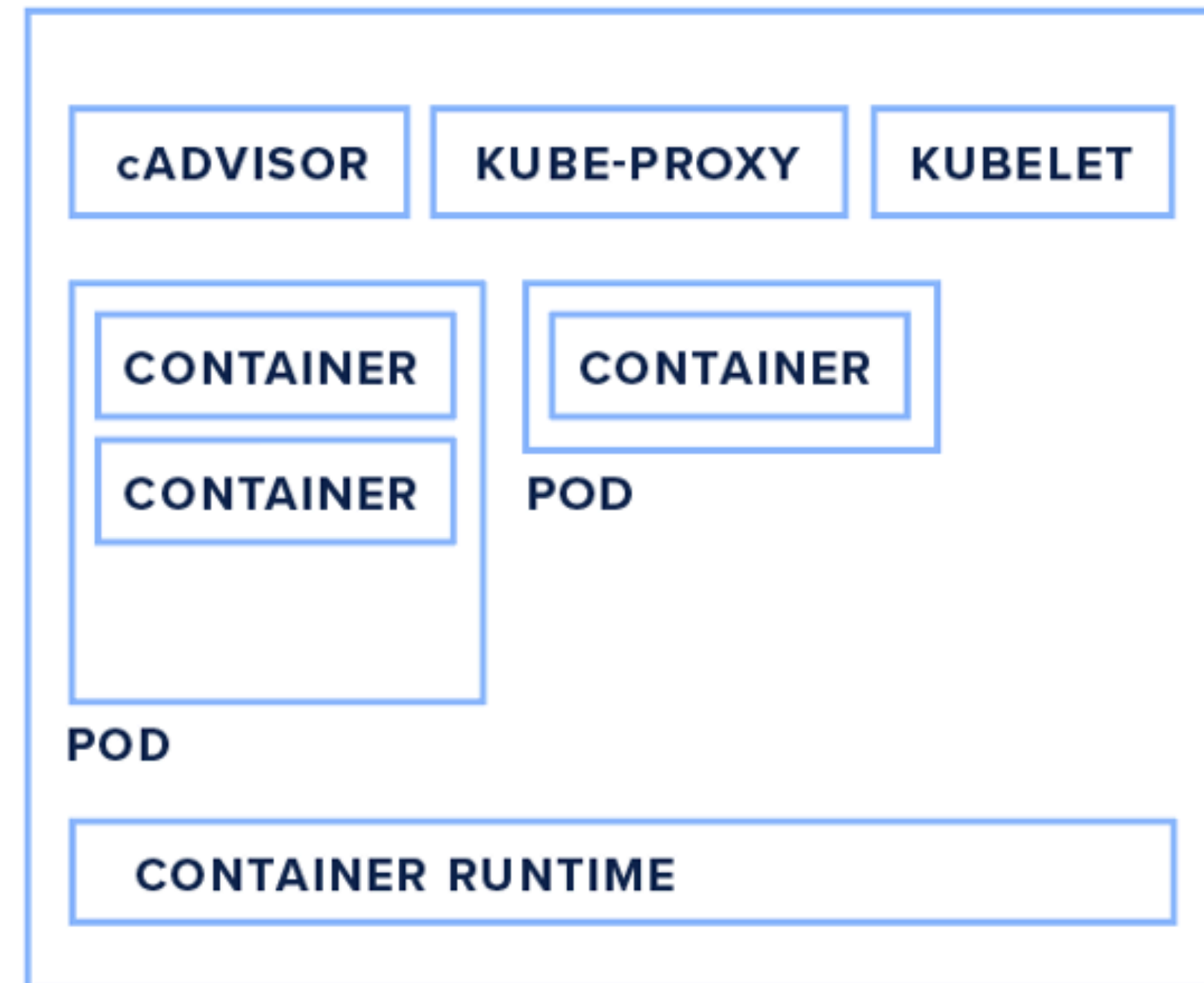
- API server
- Scheduler
- Controllers
 - Kubernetes
 - Cloud
- Etcd



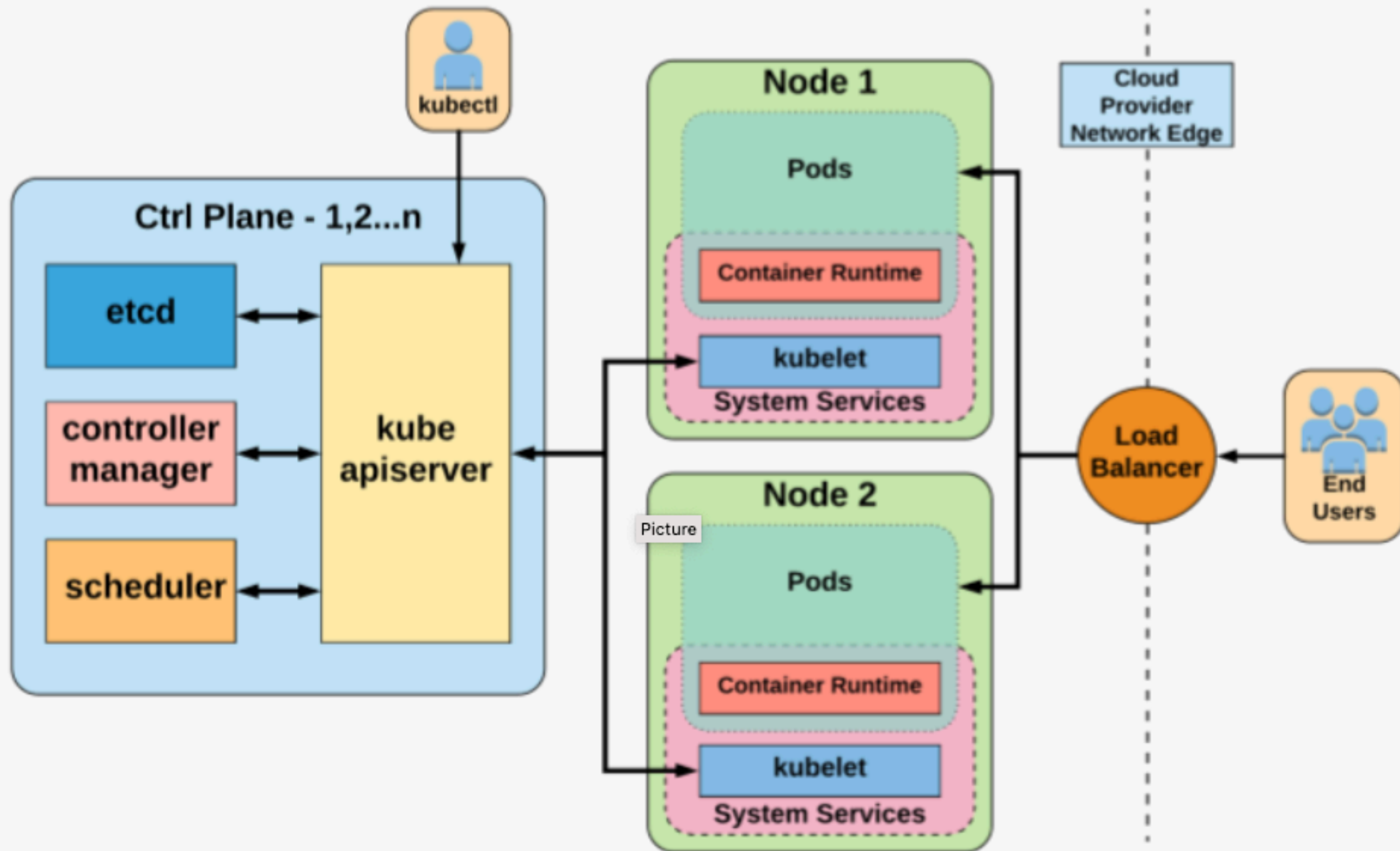
Kubernetes Architecture

- Nodes

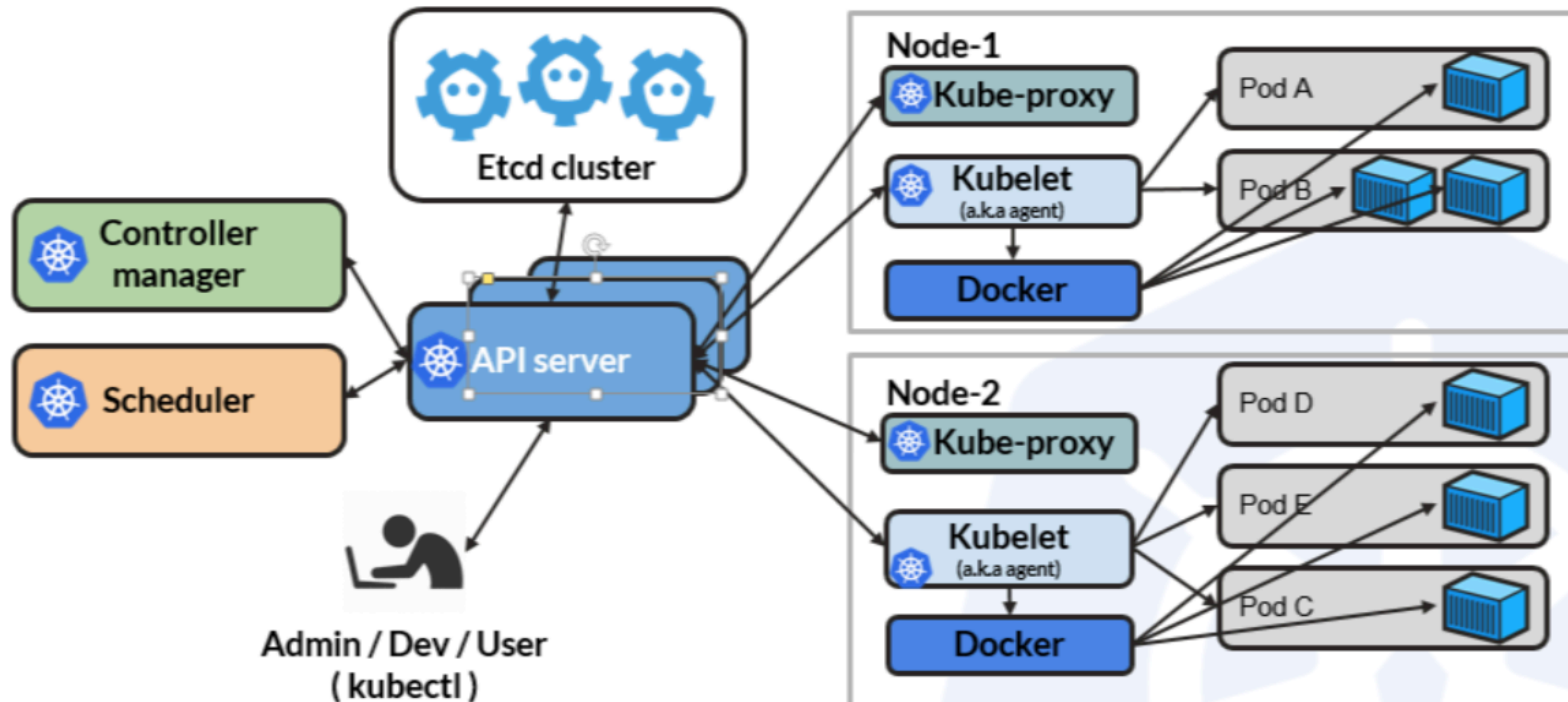
- Kubelet
- Kube-proxy
- cAdvisor
- Container runtime



Kubernetes Architecture



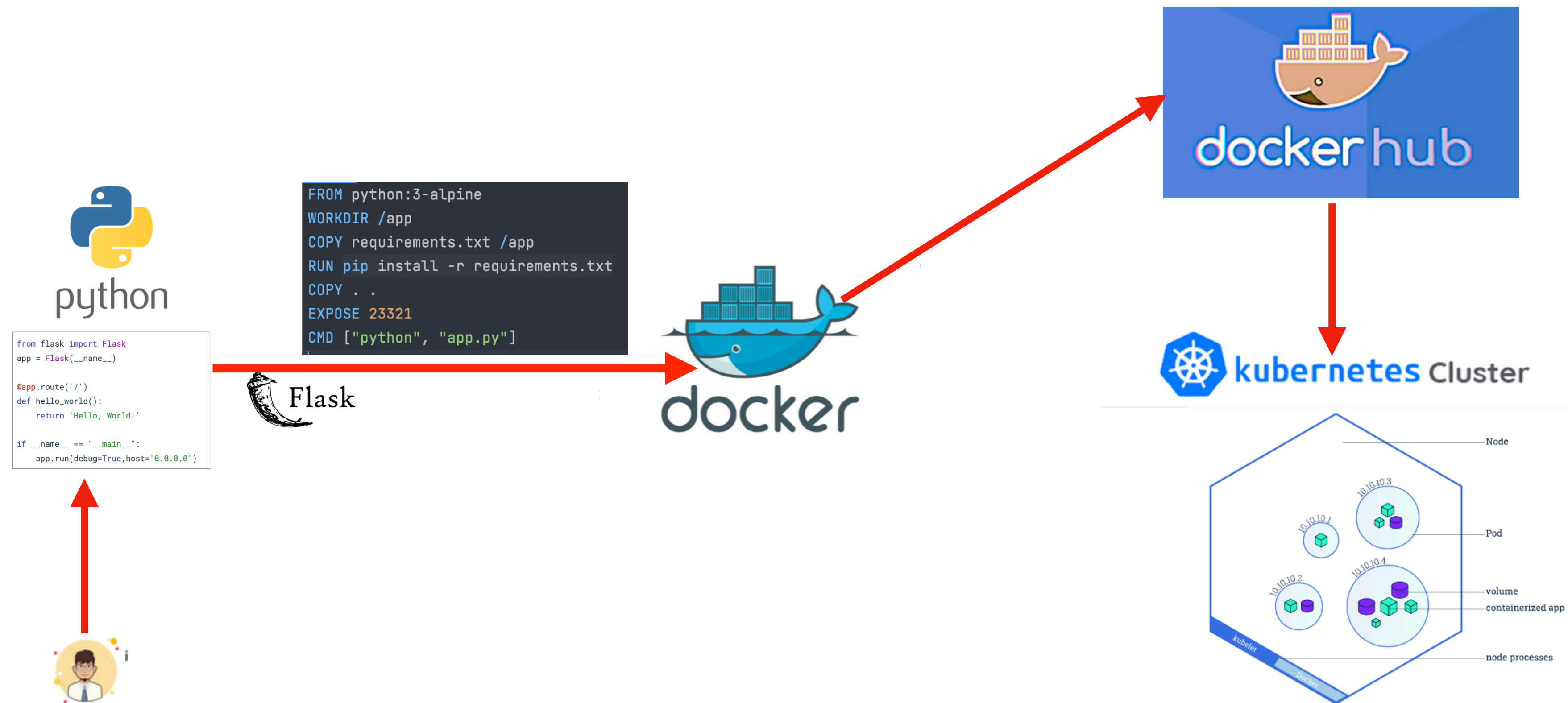
Kubernetes Architecture



How do I interact with a Kubernetes cluster?

- Hit REST API directly
 - Can use curl, client libraries, etc.
- Kubectl
 - Command-line tool to interact with control plane
 - Abstracts away multiple REST API calls
 - Provides “get” “create” “delete” “describe”, etc. functionality
 - Filtering results
- Set up kubectl
 - `cp k8s_config_file ~/.kube/config`
 - May need to create this directory, depending on your OS
 - `kubectl cluster-info`

Containerized Flask App



Kubernetes Components

Kubernetes - 7 use cases

**kubernetes**

01

Simple app
Learning how to get your app up and running by deploying it on a Kubernetes cluster

02

Microservices
Orchestrating complicated apps based on a microservice architecture with many components

03

Lift and shift
Facilitating the moving of on-prem apps to the cloud

04

Cloud Native Network Functions
Managing containers with encapsulated network functions (CNF initiatives)

05

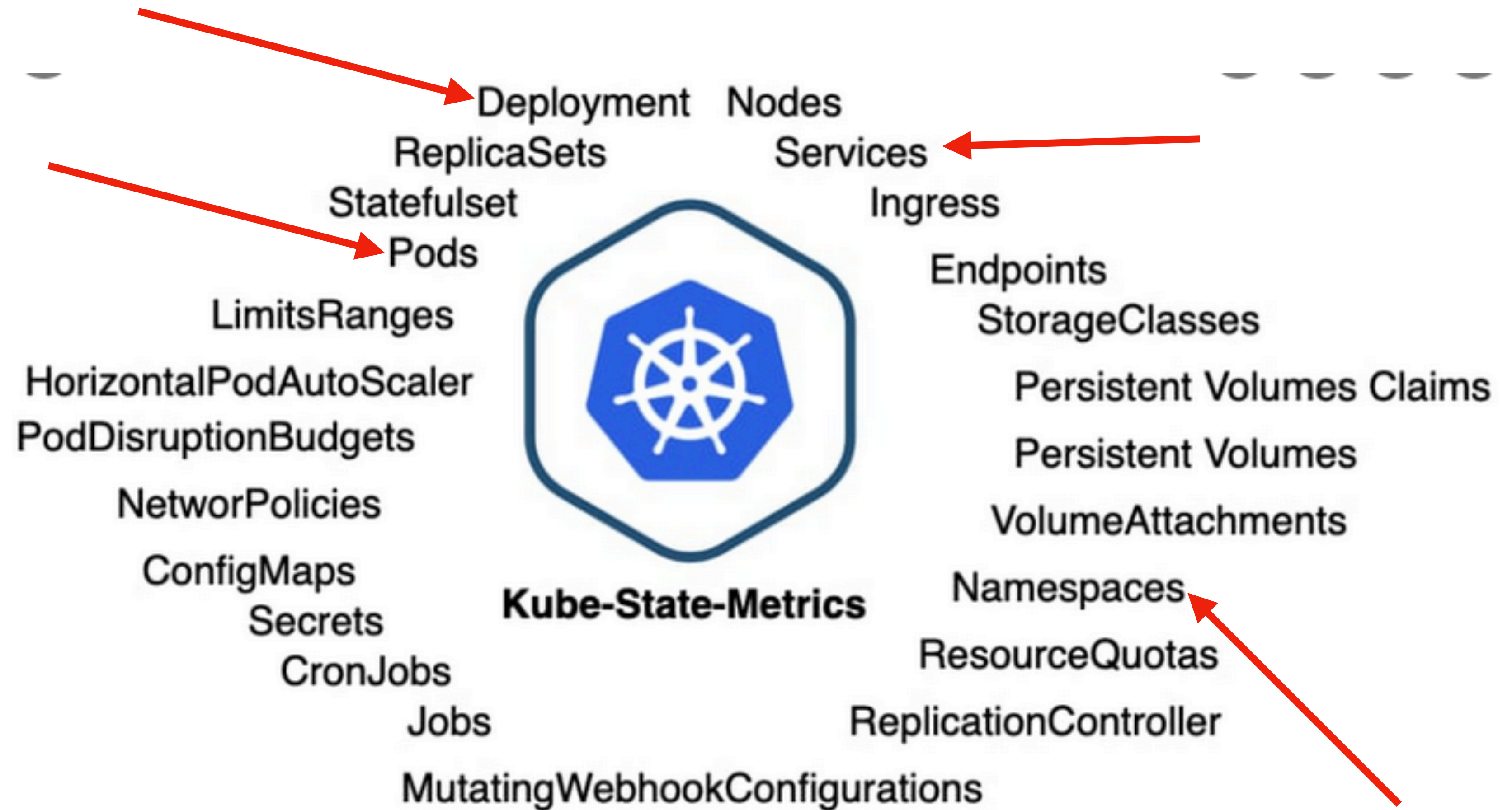
Machine Learning
Managing machine learning workflows to deploy faster AI-based apps

06

Heavy computing
Enabling computing-heavy tasks

07

CI/CD
CI/CD tasks are easier to perform with Kubernetes



- An abstraction that allows you to divide a cluster into multiple scoped “virtual clusters”
 - E.g. Each team gets its own Namespace with associated resource quota
- Primary mechanism for scoping and limiting access
- Kubernetes usually starts with 3 Namespaces by default
 - default
 - kube-system
 - kube-public

Creating a Namespace

- List namespaces with kubectl:
 - `kubectl get namespaces`
 - `kubectl get ns`
- Create your own:
 - `kubectl create ns flask`
- Specify a namespace with kubectl:
 - `kubectl -n flask get all`
- If you don't want to use the `-n` flag with every command: **contexts**
 - `kubectl config current-context`
 - `kubectl config set-context --current --namespace=flask`
 - `kubectl config get-contexts`
 - `kubectl get all`

Pods

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

- Fundamental Kubernetes work unit
- Can run one or more containers
 - Why more than one?
- Pod containers share resources
 - Storage
 - Network (localhost)
 - Always run on the same Node

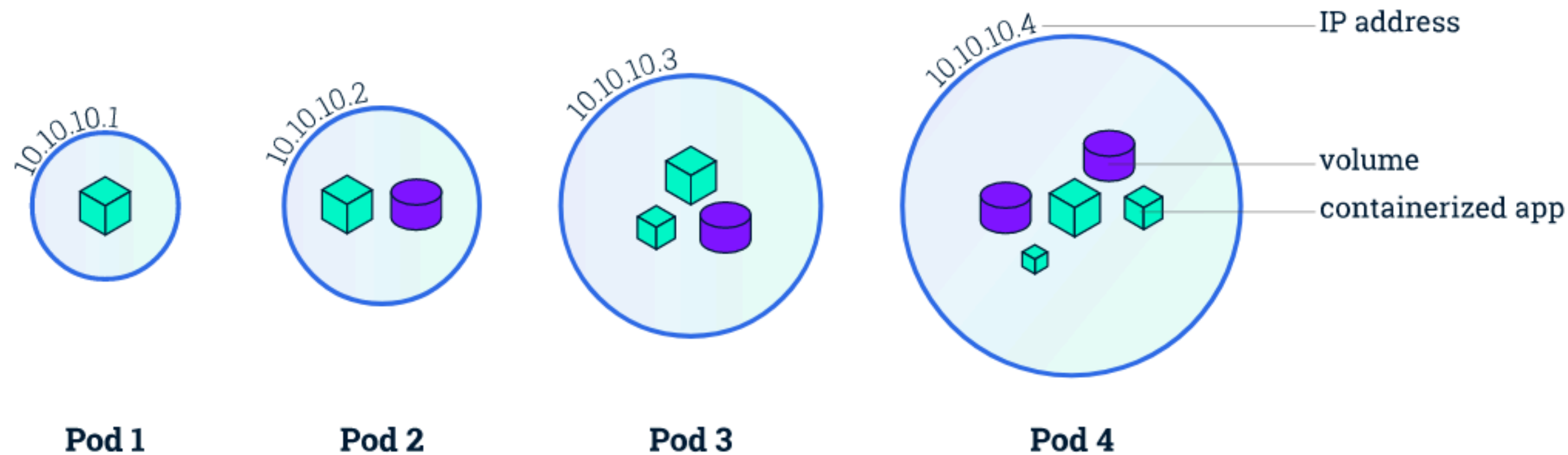
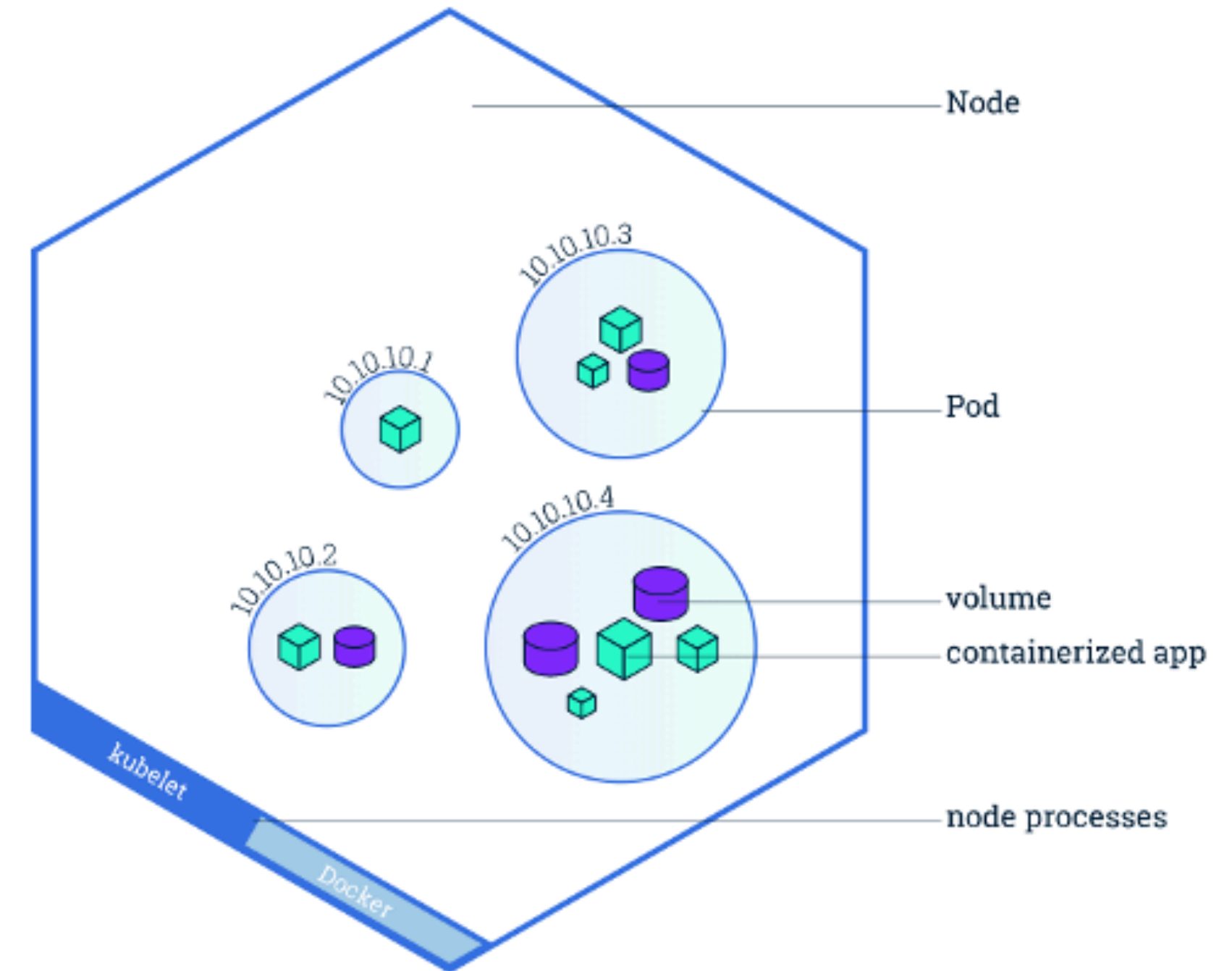
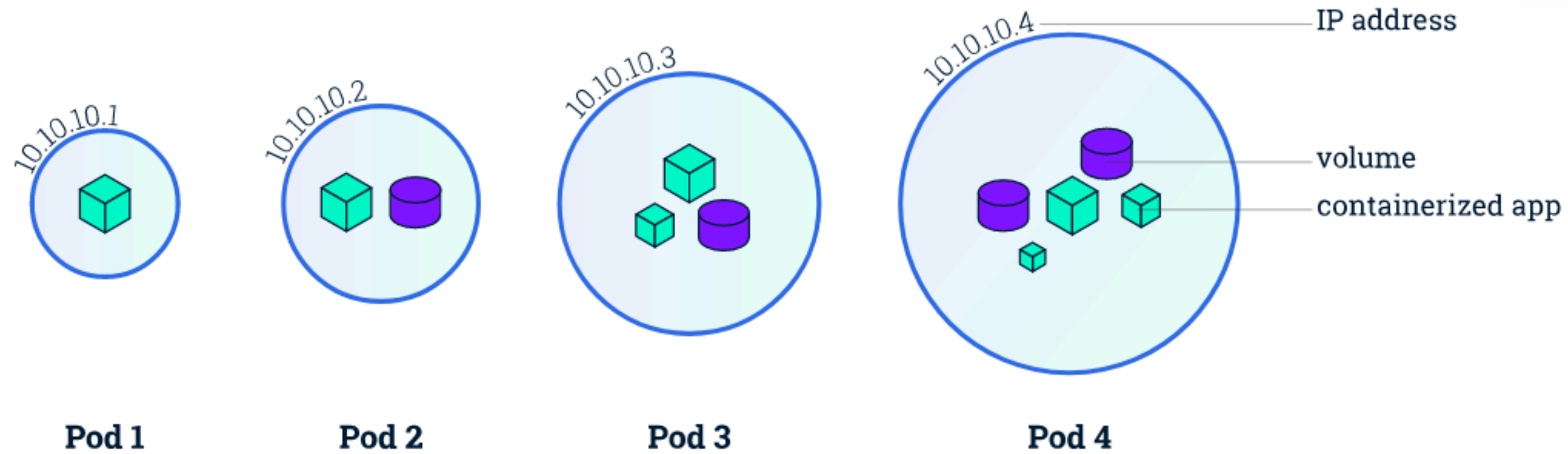


Image Attribution: [K8s Official Docs](#)

Pods

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```



Pod Manifest Example

```
apiVersion: v1
kind: Pod
metadata:
  name: flask-pod
  labels:
    app: flask-helloworld
spec:
  containers:
  - name: flask
    image: worldbosskafka/flask-helloworld:latest
    ports:
    - containerPort: 5000
```

- Deploy the Flask Pod
 - `kubectl apply -f flask_pod.yaml -n flask`
- Check that it's up
 - `kubectl get pod -n flask`
- Forward a local port into the cluster so that we can access it
 - `kubectl port-forward -n flask pods/flask-pod 5000:5000`
 - `curl http://localhost:5000`
- Delete the Pod
 - `kubectl delete pod flask-pod -n flask`

Some Kubectl Commands...

Declarative:

- kubectl get
- kubectl apply
- kubectl rollout status
- kubectl rollout undo

Imperative:

- kubectl create
- kubectl delete
- kubectl expose
- kubectl edit
- kubectl patch

Labels

- Key/value pairs: think of them as object “tags”
- Almost everything can be labeled
 - Even Nodes
- *Not* Unique
- Used to select objects with *selectors*
- Examples:
 - env: prod
 - env: staging
 - release: stable
 - release: canary

Kubernetes Workloads

- Deployments (stateless apps)
 - ReplicaSets
 - Pods
 - Containers
 - Namespaces & cgroups
- StatefulSets (stateful apps - e.g. databases)
- DaemonSets (think of these as “agents” / daemons)
- Jobs & CronJobs

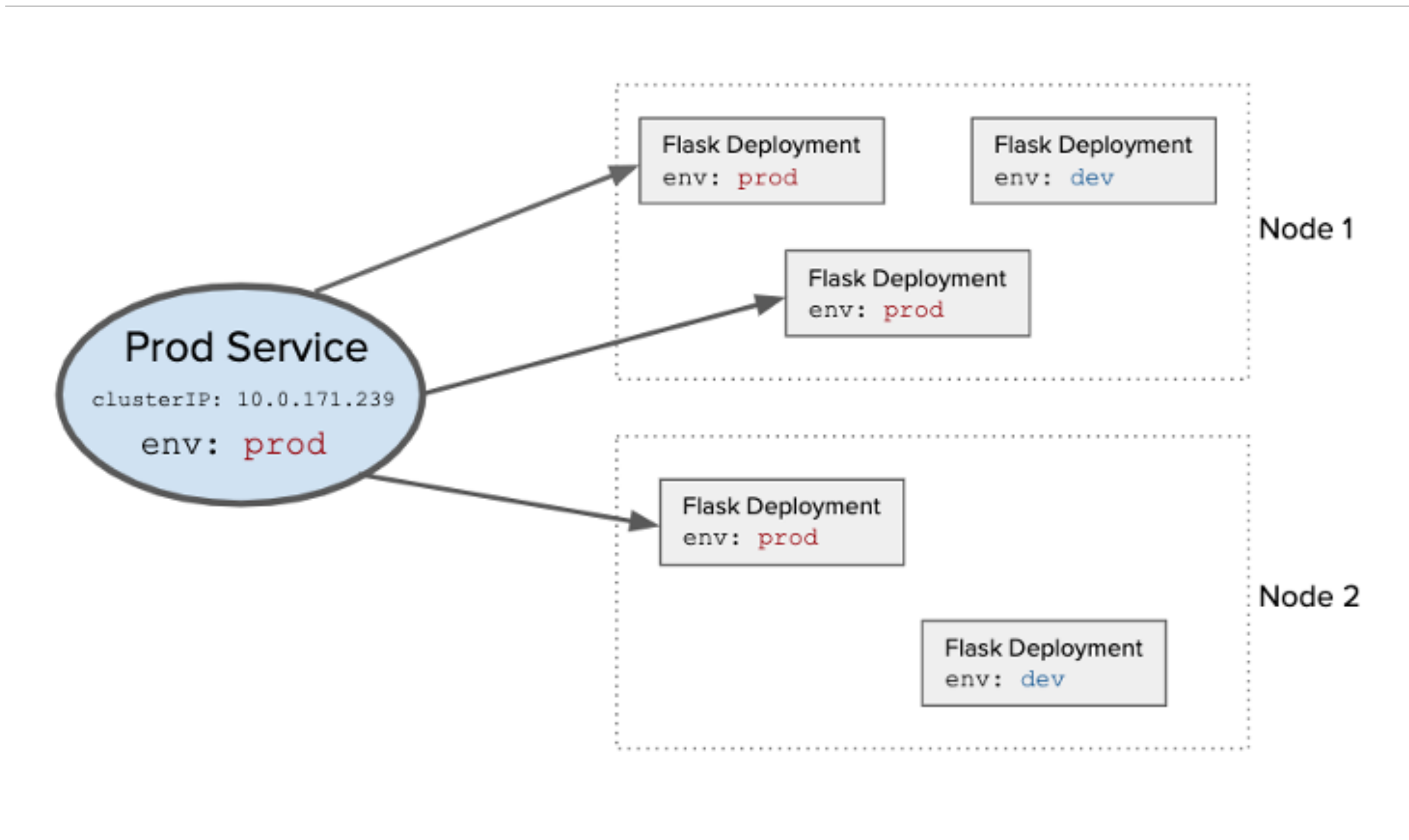
Deployments

- How to manage multiple Pods?
- Higher-level object that “contains” the Pod object
- Pod management
 - Deployment
 - Scaling
 - Updates

Services: Exposing your apps to the outside world



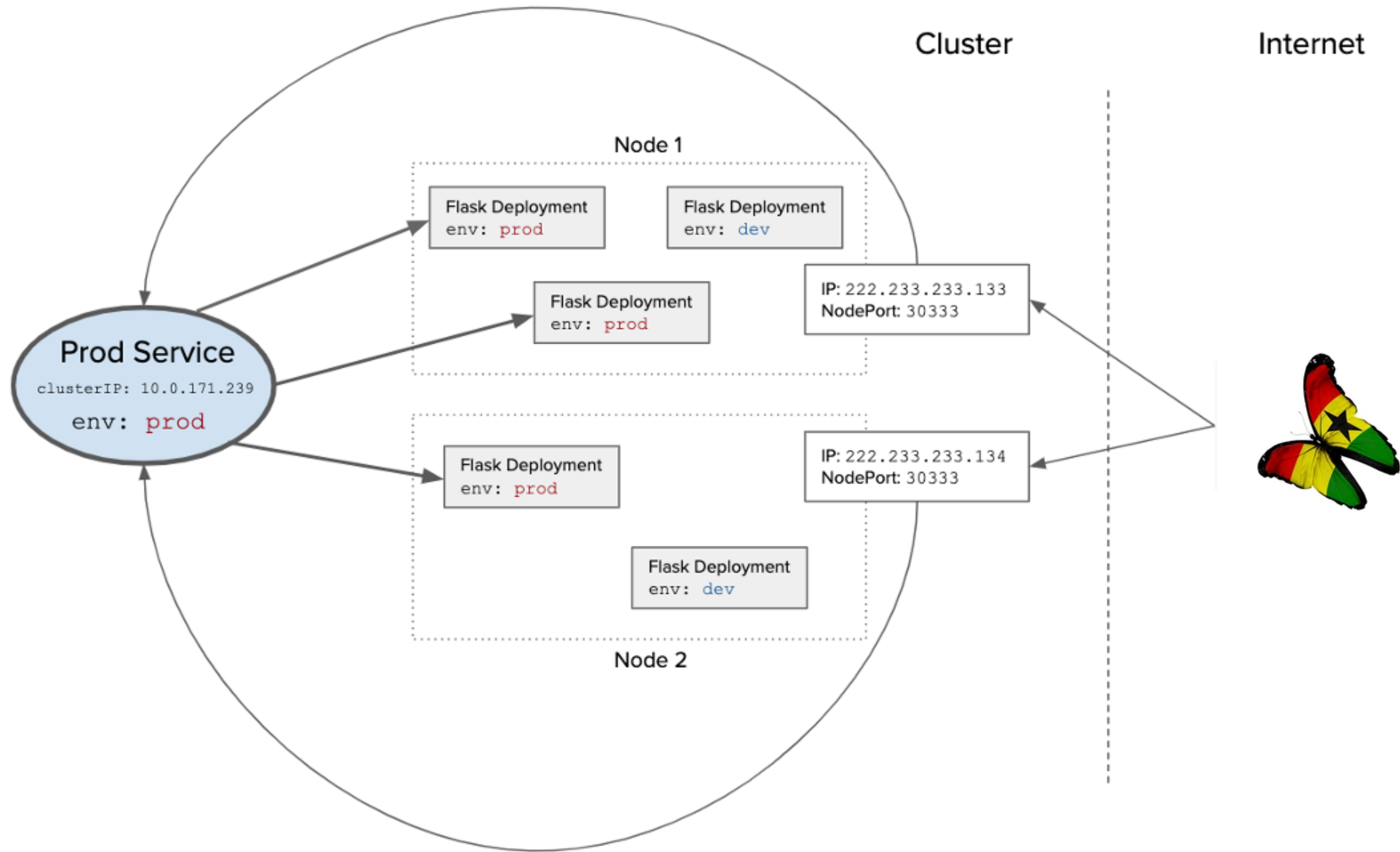
- By default, every Pod will be assigned an ephemeral cluster-internal IP address
- If you have a set of Pod replicas (Deployment), how to create a stable endpoint?
- Services: Abstraction to expose an app as a *service* (think microservices)
- Load balancing traffic
 - Routing to “healthy” / “available” Pods
- Again uses Labels + Selectors
- Example: “Prod Service”
 - ClusterIP
 - Stable network endpoint
 - Load-balances traffic to prod Deployment Pods



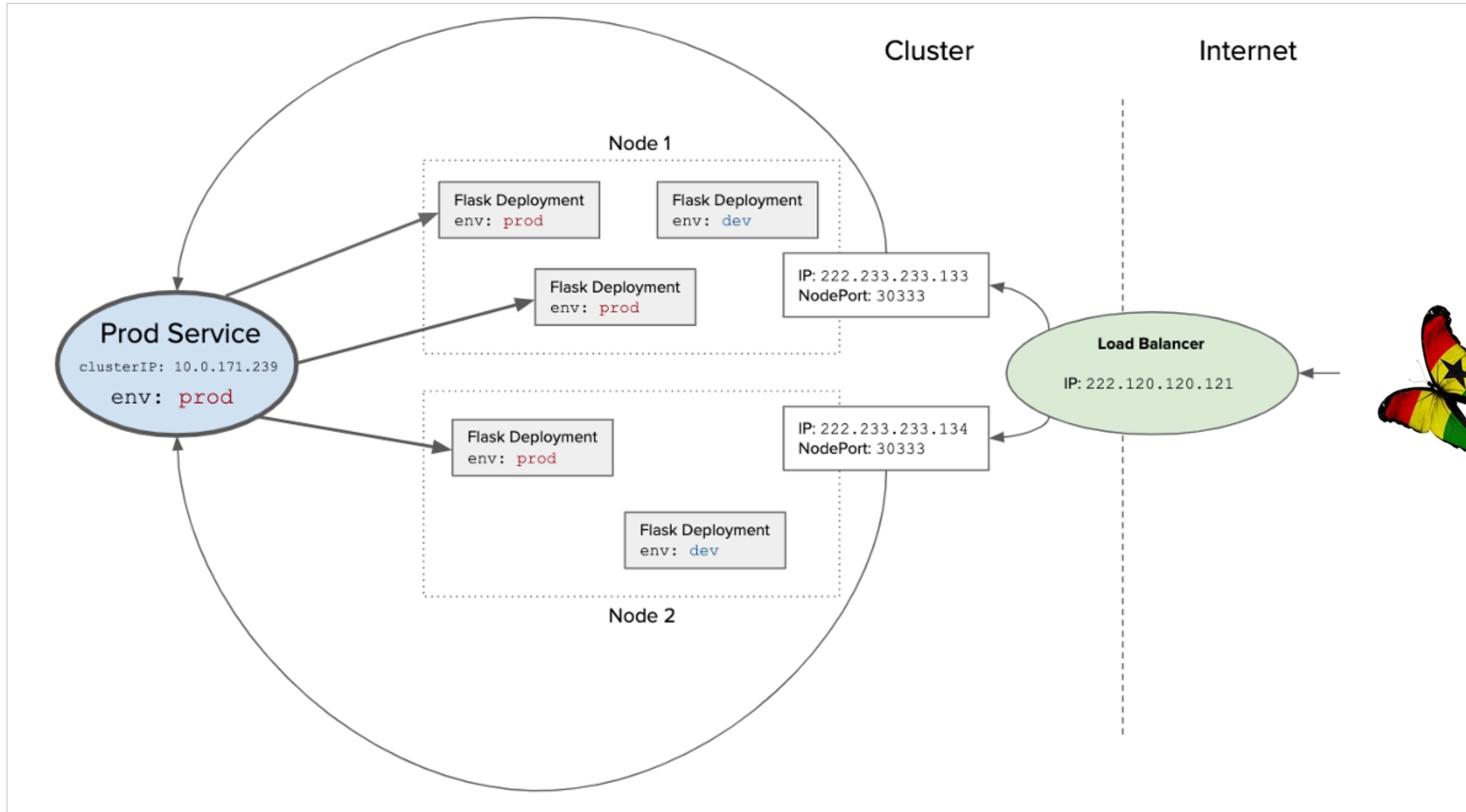
Service Types

- ClusterIP
 - Expose the service on a Cluster-internal IP
- NodePort
 - Expose the service on each Node's IP at a static port ("NodePort")
- LoadBalancer
 - Create an external LoadBalancer which routes requests to Nodeport & ClusterIP services
- Aside: Ingress Controllers

NodePort Service



LoadBalancer Service



LoadBalancer Service

```
apiVersion: v1
kind: Service
metadata:
  name: flask-svc
  labels:
    app: flask-helloworld
spec:
  type: LoadBalancer
  ports:
  - port: 80
    targetPort: 5000
    protocol: TCP
  selector:
    app: flask-helloworld
```

- [Deploy the Flask LoadBalancer Service](#)
 - `kubectl apply -f flask-service.yaml -n flask`
- Check that it's up (may have to wait for external IP)
 - `kubectl get svc -n flask`
 - `curl loadbalancer_external_ip`
- Get external IPs of Nodes (for NodePort services)
 - `kubectl get node -o wide`

Configuration: ConfigMaps & Secrets



- Kubernetes provides various features for externalizing and versioning config parameters
 - Stored in etcd
- ConfigMaps
 - Hostnames, runtime parameters for commands, config files
- Secrets
 - Base64-encoded, encrypted
 - Passwords, credentials, etc.
- Versatile, can be created and used in a number of ways
 - Env vars
 - Mounted as Volumes attached to Pods

Storage & Volumes

- Volumes
 - Tied to the lifecycle of the Pod that requests it
 - Can be used to share data between containers in a Pod
- Persistent Volumes & PVCs
 - Abstraction that allows operators to separate storage provisioning from consumption
 - For example:
 - A PV could be a 10Gi DO block storage disk made available to the cluster
 - The PVC (defined in the workload manifest) states that this particular app needs a 10Gi disk. A controller matches the PVC with the PV
- Storage Classes

More K8s Features

- Resource requests & limits
- Autoscaling
- Node affinity, taints, tolerations
- Dashboard
- Metrics-server

Helm: K8s “Package Manager”



- Tool for managing Kubernetes applications
 - Think “apt-get” for Ubuntu / package managers
- Architecture
 - Helm (client)
 - Tiller (server, runs in the cluster)
- How it works
 - Charts
 - `helm install stable/wordpress`
- Sample apps: Wordpress, Prometheus, MySQL, Drupal, ...

Where to go from here?

- [Kubernetes For Fullstack Developers Curriculum](#)
- [Kubernetes White Paper](#)
- [DigitalOcean Kubernetes Community Tutorials](#)
- [Kubernetes Official Documentation](#)
- [Kubernetes GitHub Project](#)
- [K9s](#)

<https://kubernetes.io/docs/reference/kubectl/cheatsheet/>

Where to go from here?

https://docs.google.com/spreadsheets/d/10NItoF_6y3mBwUzQ4bcQLQfCE1BWSgUDcJXy-Qp2JEU/edit#gid=0



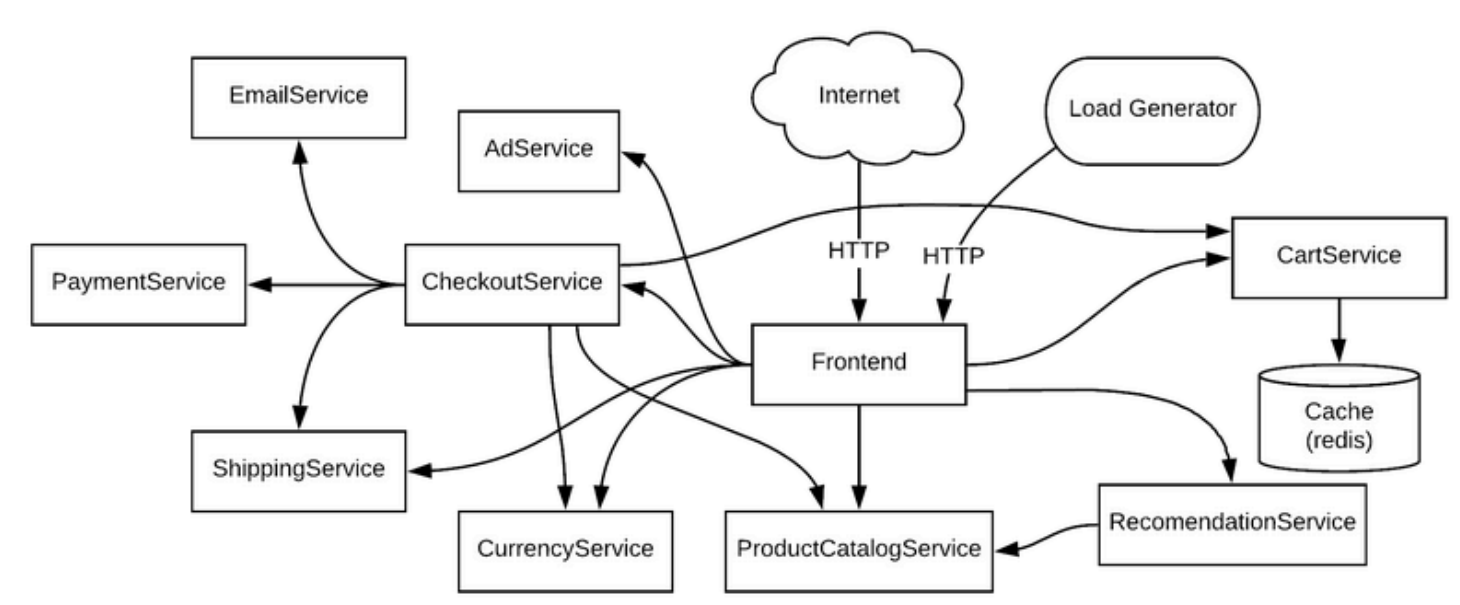
MicroServices

Microservices <https://onlineboutique.dev/>

Online Boutique is a cloud-native microservices demo application. Online Boutique consists of a 10-tier microservices application. The application is a web-based e-commerce app where users can browse items, add them to the cart, and purchase them.

Architecture

Online Boutique is composed of 11 microservices written in different languages that talk to each other over gRPC. See the [Development Principles](#) doc for more information.

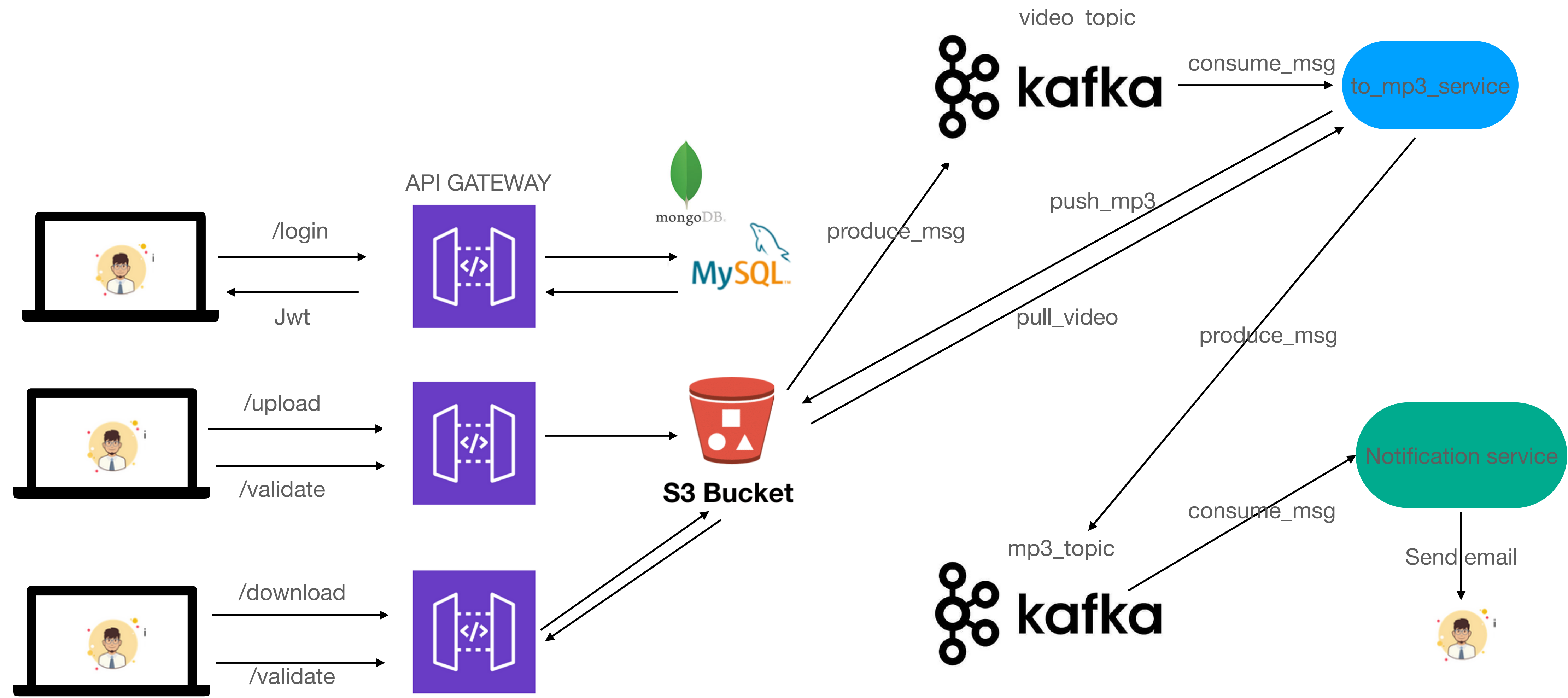


Features

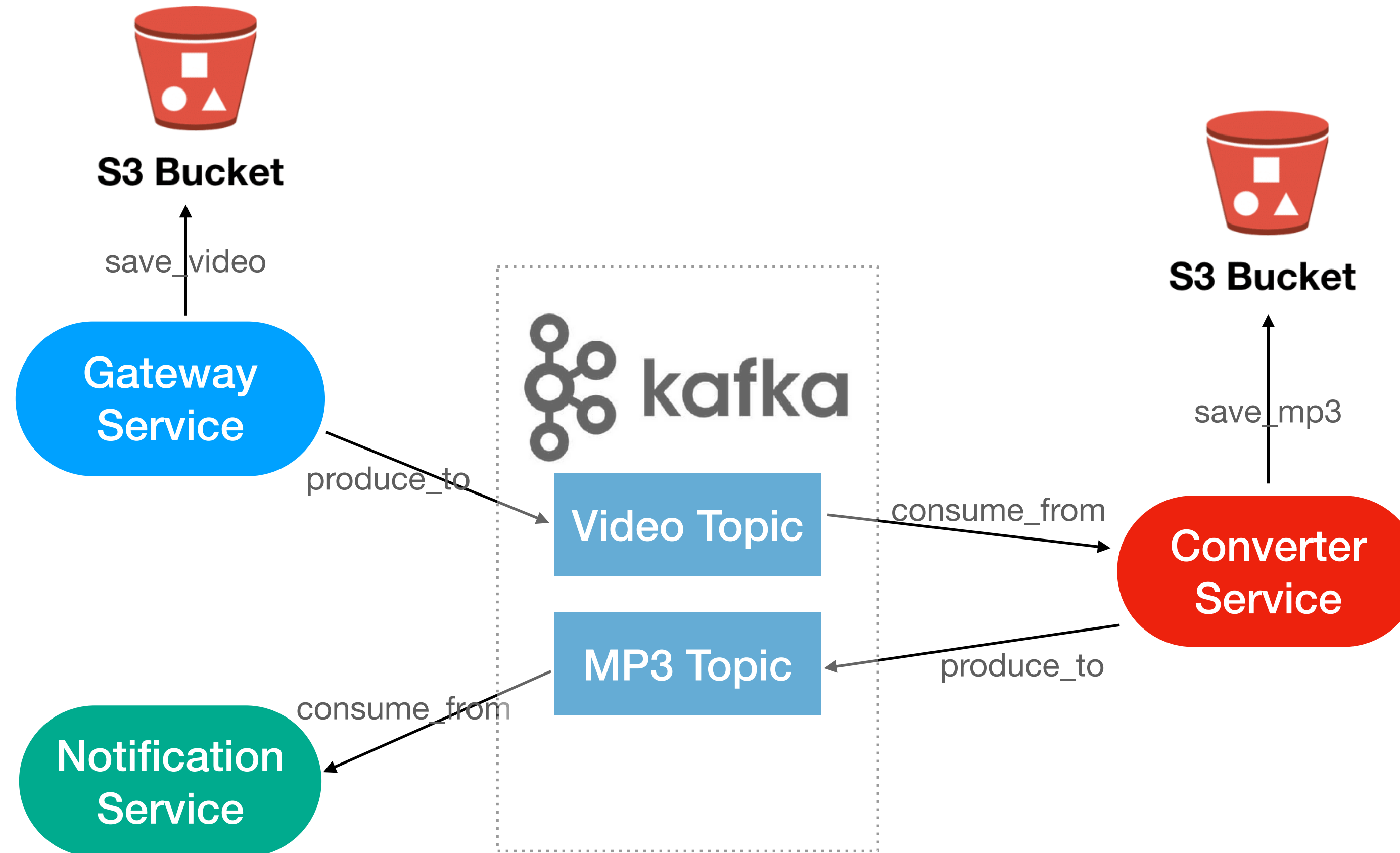
- **Kubernetes/GKE:** The app is designed to run on Kubernetes (both locally on "Docker for Desktop", as well as on the cloud with GKE).
- **gRPC:** Microservices use a high volume of gRPC calls to communicate to each other.
- **Istio:** Application works on Istio service mesh.
- **OpenCensus Tracing:** Most services are instrumented using OpenCensus trace interceptors for gRPC/HTTP.
- **Cloud Operations (Stackdriver):** Many services are instrumented with **Profiling**, **Tracing** and **Debugging**. In addition to these, using Istio enables features like Request/Response **Metrics** and **Context Graph** out of the box. When it is running out of Google Cloud, this code path remains inactive.
- **Scaffold:** Application is deployed to Kubernetes with a single command using Scaffold.
- **Synthetic Load Generation:** The application demo comes with a background job that creates realistic usage patterns on the website using [Locust](#) load generator.

Service	Language	Description
frontend	Go	Exposes an HTTP server to serve the website. Does not require signup/login and generates session IDs for all users automatically.
cartservice	C#	Stores the items in the user's shopping cart in Redis and retrieves it.
productcatalogservice	Go	Provides the list of products from a JSON file and ability to search products and get individual products.
currencyservice	Node.js	Converts one money amount to another currency. Uses real values fetched from European Central Bank. It's the highest QPS service.
paymentservice	Node.js	Charges the given credit card info (mock) with the given amount and returns a transaction ID.
shippingservice	Go	Gives shipping cost estimates based on the shopping cart. Ships items to the given address (mock)
emailservice	Python	Sends users an order confirmation email (mock).
checkoutservice	Go	Retrieves user cart, prepares order and orchestrates the payment, shipping and the email notification.
recommendationservice	Python	Recommends other products based on what's given in the cart.
adservice	Java	Provides text ads based on given context words.
loadgenerator	Python/Locust	Continuously sends requests imitating realistic user shopping flows to the frontend.

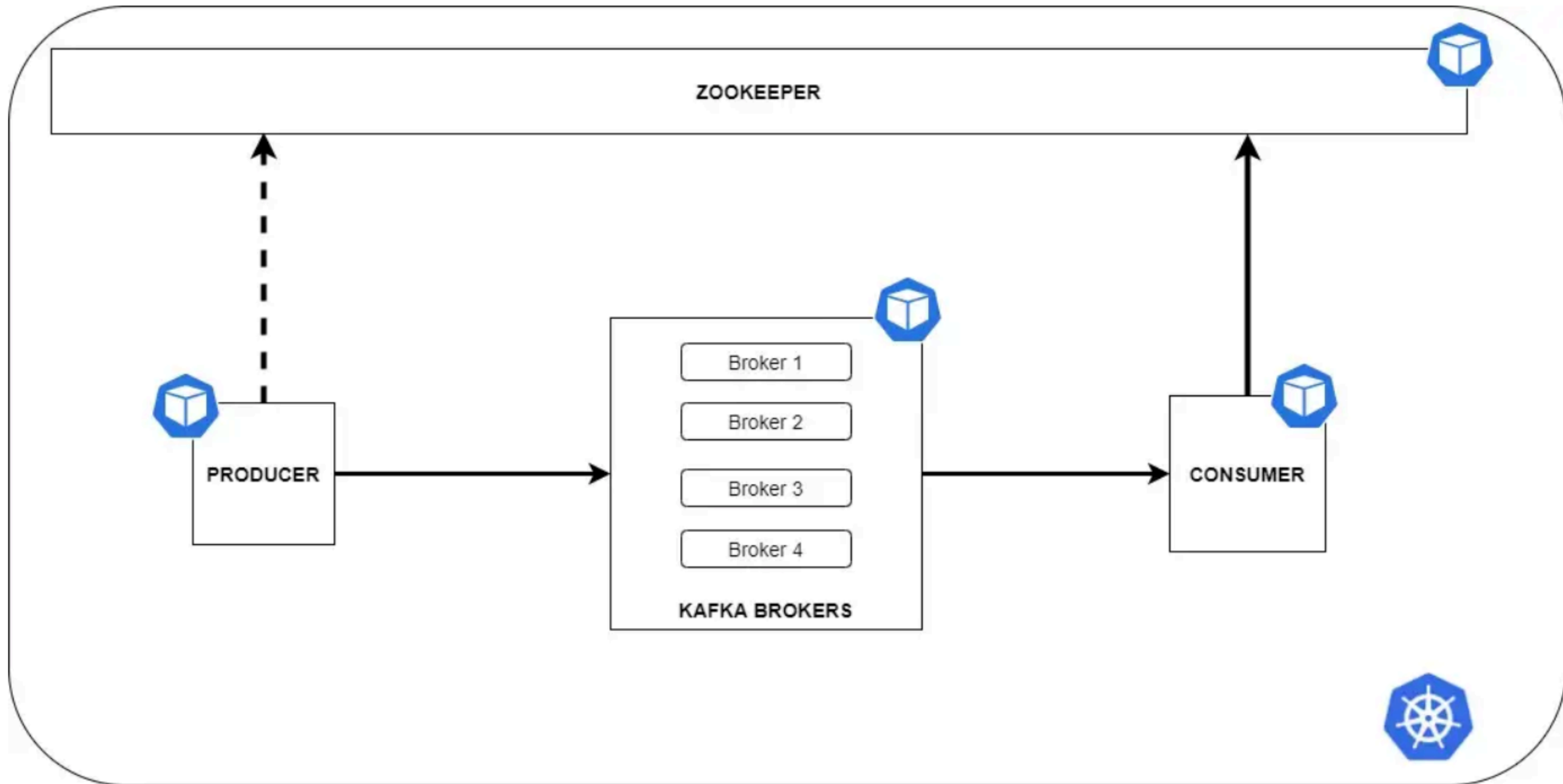
MicroService With Python



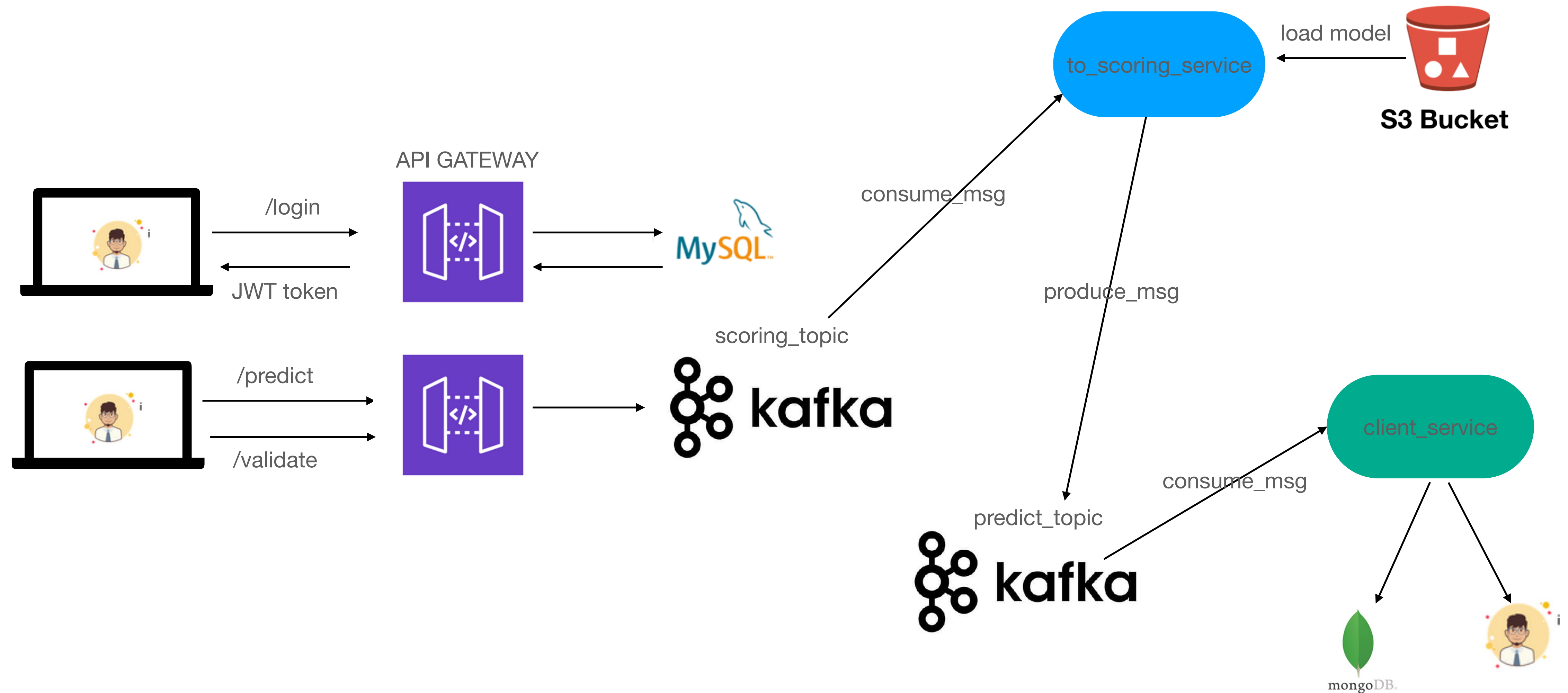
MicroService With Python



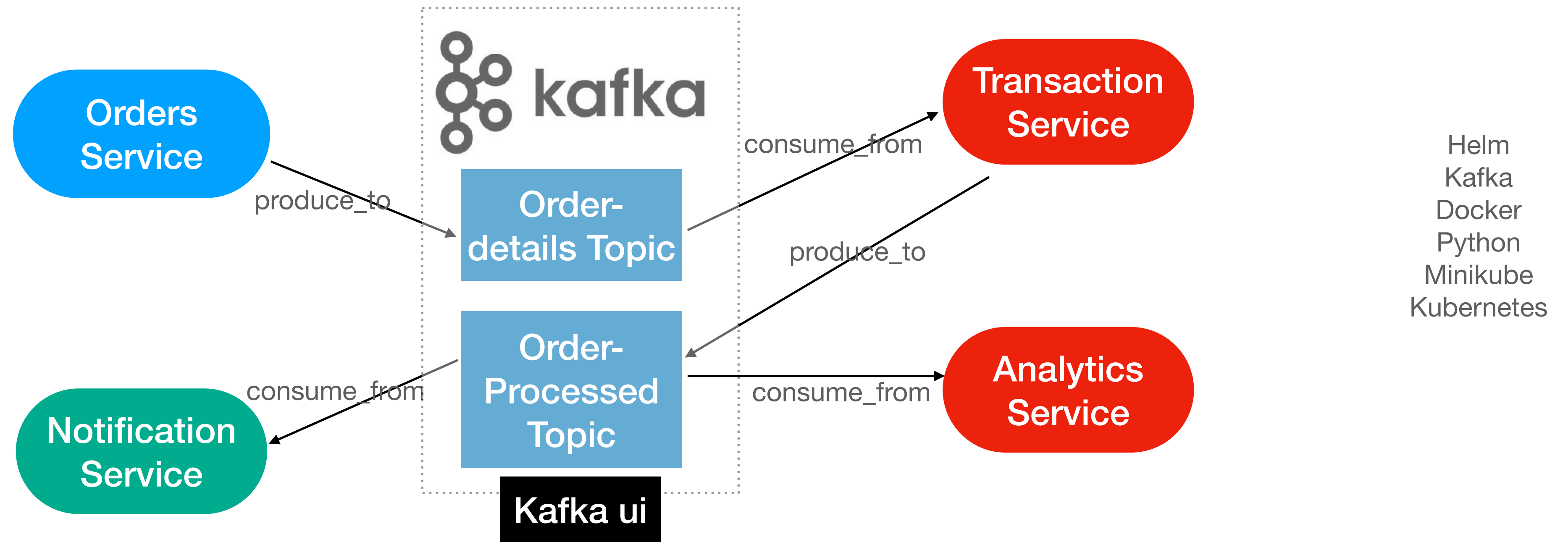
Kafka Kubernetes MicroService



Machine Learning MicroService With Python

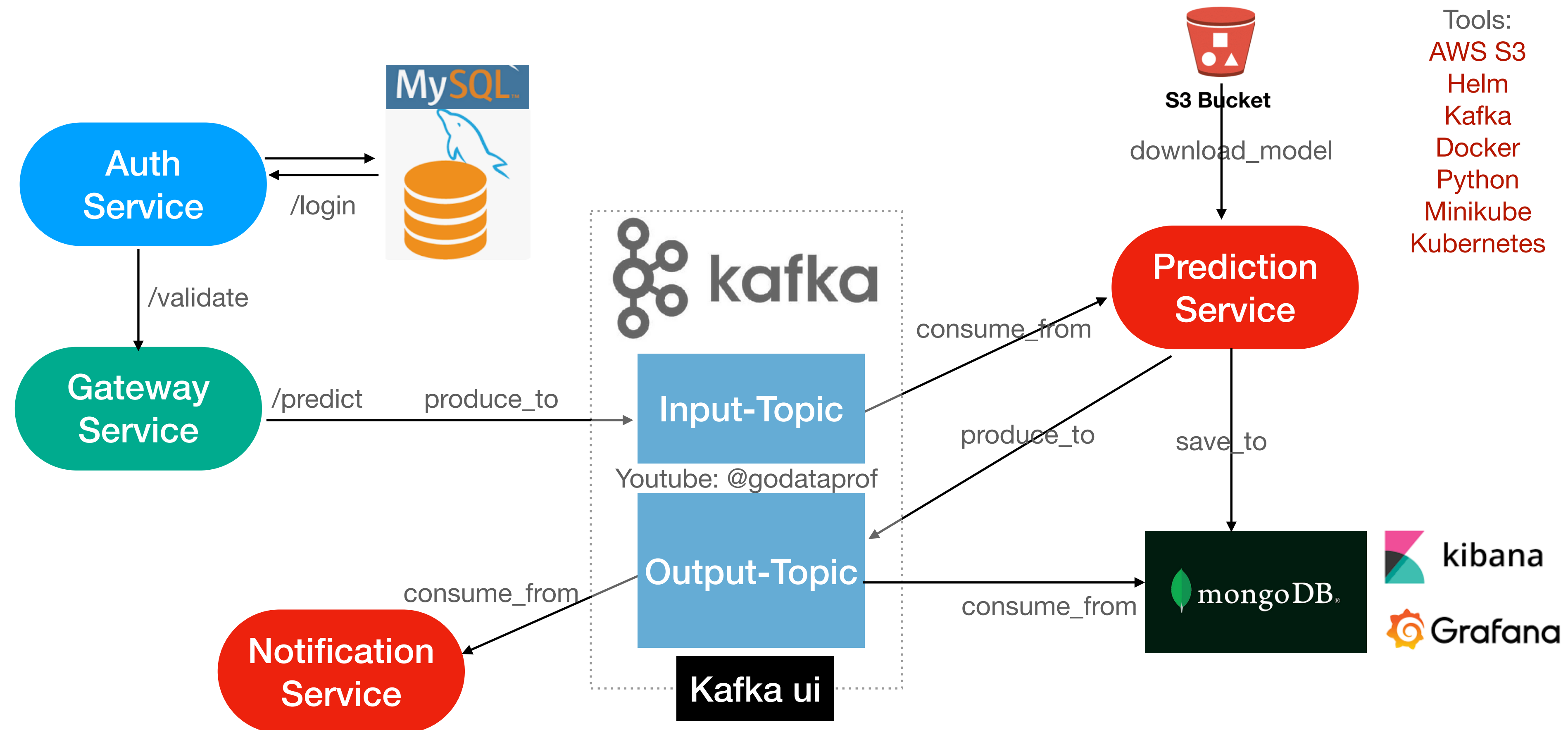


Orders MicroService With Python & Kafka



Youtube: @godataprof

Near Real-Time Machine Learning MicroService



Python Application to Email Current Bitcoin Price



Bitcoin Service

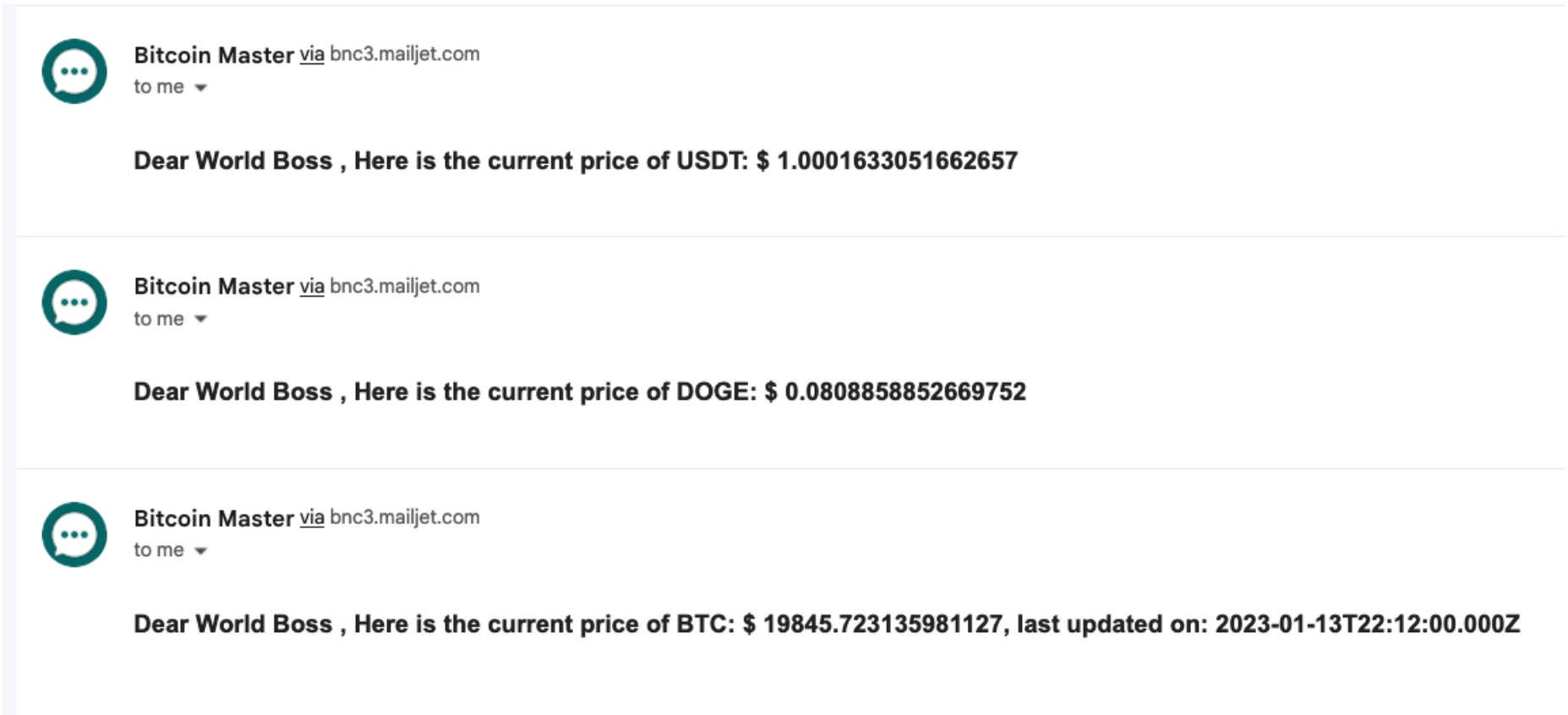
Data:
<https://coinmarketcap.com>
Email:
<https://mailjet.com>

Tools:
Docker
Python
Minikube
Kubernetes

Email Service

`http://0.0.0.0:1957/api/v1.0/sendEmail`

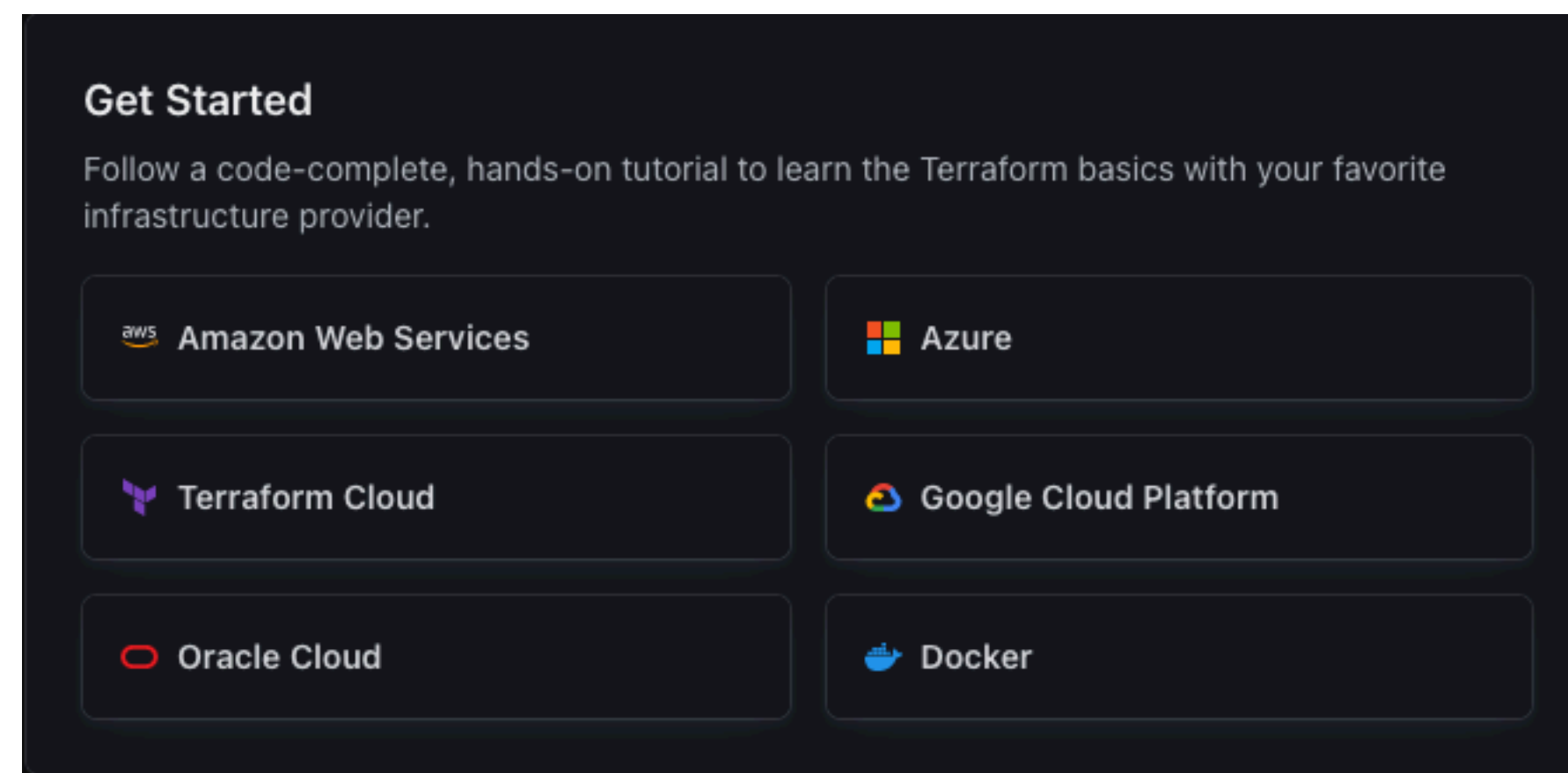
```
1 {  
2   ... "name": "World Boss",  
3   ... "email": "user.aerogramme@gmail.com",  
4   ... "symbol": "BTC"  
5 }  
6 |
```



Terraforms

Terraforms Basics

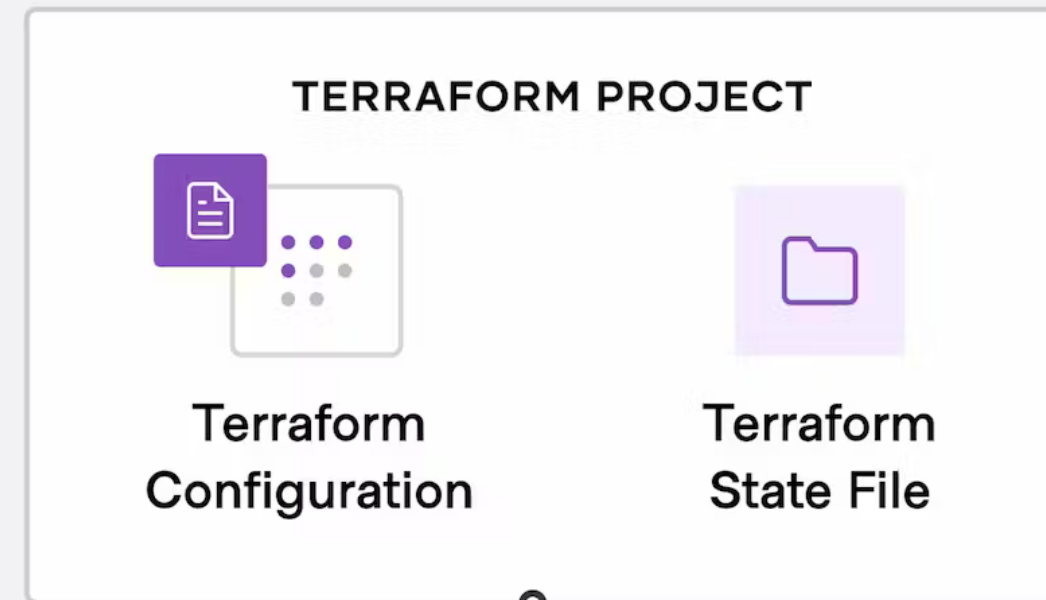
- **Terraform** is an infrastructure as code tool that lets you build, change, and version infrastructure safely and efficiently.
- This includes low-level components like compute instances, storage, and networking; and high-level components like DNS entries and SaaS features.



Terraforms Basics

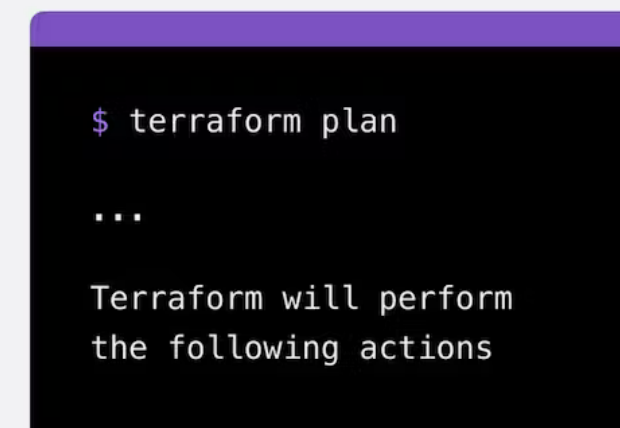
Write

Define infrastructure in configuration files



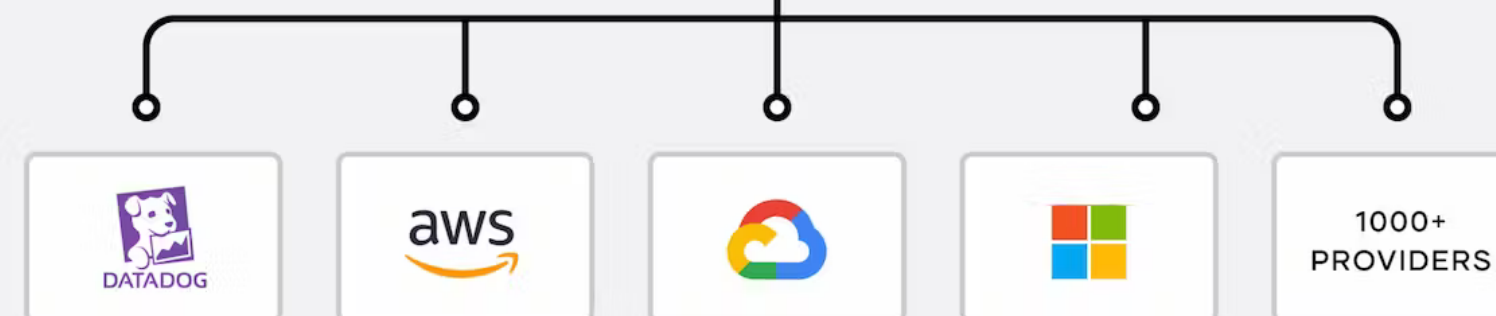
Plan

Review the changes
Terraform will make to
your infrastructure



Apply

Terraform provisions
your infrastructure and
updates the state file.



Terraforms Commands

The available commands for execution are listed below.
The primary workflow commands are given first, followed by
less common or more advanced commands.

Main commands:

init	Prepare your working directory for other commands
validate	Check whether the configuration is valid
plan	Show changes required by the current configuration
apply	Create or update infrastructure
destroy	Destroy previously-created infrastructure

All other commands:

console	Try Terraform expressions at an interactive command prompt
fmt	Reformat your configuration in the standard style
force-unlock	Release a stuck lock on the current workspace
get	Install or upgrade remote Terraform modules
graph	Generate a Graphviz graph of the steps in an operation
import	Associate existing infrastructure with a Terraform resource
login	Obtain and save credentials for a remote host
logout	Remove locally-stored credentials for a remote host
metadata	Metadata related commands
output	Show output values from your root module
providers	Show the providers required for this configuration
refresh	Update the state to match remote systems
show	Show the current state or a saved plan
state	Advanced state management
taint	Mark a resource instance as not fully functional
test	Experimental support for module integration testing
untaint	Remove the 'tainted' state from a resource instance
version	Show the current Terraform version
workspace	Workspace management

Global options (use these before the subcommand, if any):

-chdir=DIR	Switch to a different working directory before executing the given subcommand.
-help	Show this help output, or the help for a specified subcommand.
-version	An alias for the "version" subcommand.

Projects

- Deploy a simple python flask app to production
- Deploy a simple machine learning model to production
- Deploy a simple image detection model to production
- Deploy a simple java application to production
- Deploy a simple NodeJS restful API to production
- Deploy a simple website to production

Class Info

Duration: 6months (slow pace) | 3months (faster pace)
Meetings: 3hours per week.
Cost: \$3,000 but 50% discount for ...
Requirements: VSCode, PyCharm, IntelliJIdea, JupyterNotebook, Docker.

Resume Prep and Interview: \$100 fee. I can help with how to pass the Cloud Certifications (AWS/Azure).

Payment Plan:

- \$500 1st down payment
- \$500 2nd payment (next 3months)
- \$500 3rd payment (next 3months)

Location: Teams

Time: Tuesday 1pm-4pm EST | 2pm-5pm CDT